

Higher-order Feature Learning

Building a computer vision “swiss army knife”

Roland Memisevic

Univ. of Frankfurt

DAGM 2011

Higher-order Feature Learning

Tutorial Website:

<http://www.cs.toronto.edu/~rfm/relational/index.html>

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

- Applications
- Conclusions

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

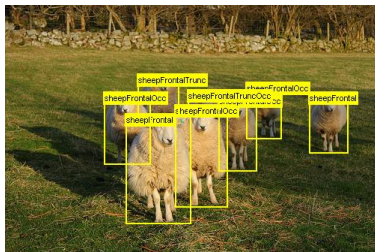
- Applications
- Conclusions

What this is about

- Extending feature learning to model *relations*.
- “Bi-linear models”, “energy-models”, “complex cells”, “spatio-temporal features”, “covariance features”, “bi-linear classification”, “mapping units”, ...

Recognition tasks

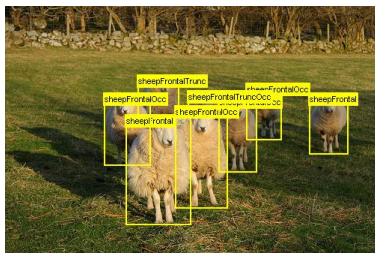
- Recognition has become a focus of interest in Computer Vision.
- Recognition of static objects started to work *very* well. In fact –
- Recognition is getting quite serious.



(PASCAL challenge)

“It’s the feature, stupid!”

- One contributor is the use of the *right representation*:
- Many recognition tasks disappeared after the community converged on the right kinds of *features*.
- With the right *representation*, the choice of top level classifier (SVM, logreg, NN) doesn’t matter all that much.
- Features are *local*, so occlusions do not matter.



(PASCAL challenge)

Recognition with local features



- Task: Recognize the building.
- Two approaches:
 - 1 Bag-Of-Features
 - 2 Convolution

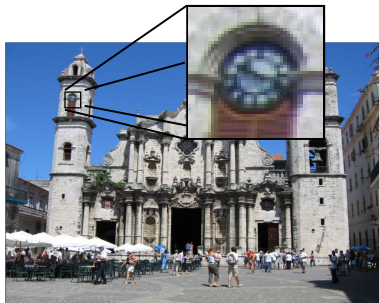
Bag-Of-Features



Bag-Of-Features

- 1 Find **interest points** (AKA keypoints).
- 2 Crop patches around interest points.
- 3 Represent each patch with a **sparse local descriptor** ("features").
- 4 **Add** all local descriptors to obtain a global descriptor for the image.

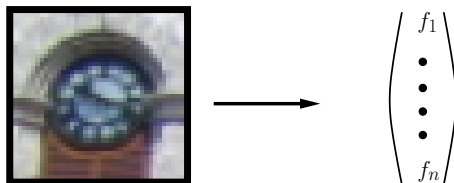
Bag-Of-Features



Bag-Of-Features

- 1 Find **interest points** (AKA keypoints).
- 2 Crop patches around interest points.
- 3 Represent each patch with a **sparse local descriptor** ("features").
- 4 **Add** all local descriptors to obtain a global descriptor for the image.

Bag-Of-Features



Bag-Of-Features

- 1 Find **interest points** (AKA keypoints).
- 2 Crop patches around interest points.
- 3 Represent each patch with a **sparse local descriptor** (“features”).
- 4 **Add** all local descriptors to obtain a global descriptor for the image.

Bag-Of-Features

$$\begin{pmatrix} f_1^1 \\ \vdots \\ f_n^1 \end{pmatrix} + \dots + \begin{pmatrix} f_1^M \\ \vdots \\ f_n^M \end{pmatrix}$$

Bag-Of-Features

- 1 Find **interest points** (AKA keypoints).
- 2 Crop patches around interest points.
- 3 Represent each patch with a **sparse local descriptor** (“features”).
- 4 **Add** all local descriptors to obtain a global descriptor for the image.

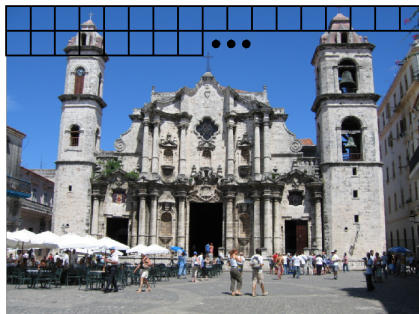
Bag-Of-Features

$$\begin{pmatrix} f_1^1 \\ \vdots \\ f_n^1 \end{pmatrix} + \dots + \begin{pmatrix} f_1^M \\ \vdots \\ f_n^M \end{pmatrix}$$

Bag-Of-Features

- 1 Find **interest points** (AKA keypoints).
- 2 Crop patches around interest points.
- 3 Represent each patch with a **sparse local descriptor** (“features”).
- 4 **Add** all local descriptors to obtain a global descriptor for the image.

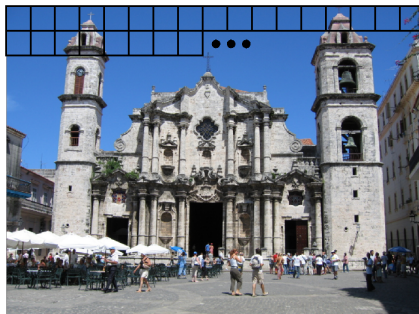
Convolutional



Convolutional

- 1 Crop patches along a regular grid (dense or not).
- 2 Represent each patch with a local descriptor.
- 3 Concatenate all descriptors into a very large vector.

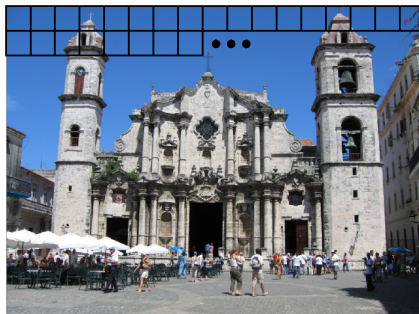
Convolutional



Convolutional

- 1 Crop patches along a regular grid (dense or not).
- 2 **Represent each patch with a local descriptor.**
- 3 **Concatenate** all descriptors into a very large vector.

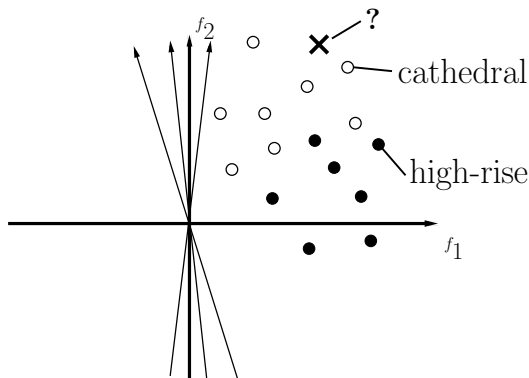
Convolutional



Convolutional

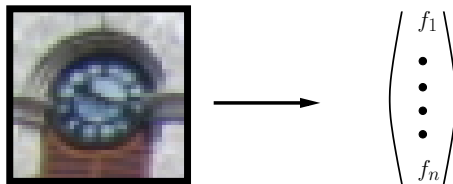
- 1 Crop patches along a regular grid (dense or not).
- 2 **Represent each patch with a local descriptor.**
- 3 **Concatenate** all descriptors into a very large vector.

Classification



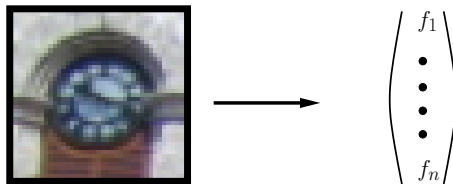
- When images are represented as points in $\mathbb{R}^n$, we can use a simple classifier to do recognition.
- Eg., Logistic regression, SVM, NN, ...
- (There are various extensions, like fancy pooling, etc.)

Feature Learning



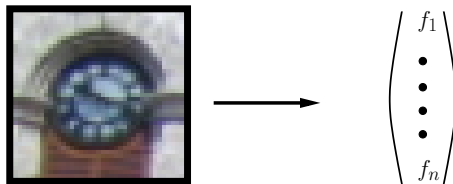
- How do we get good features?
- Option B: Engineer them. SIFT, HOG, LBP, etc.
- But: Natural Images are not Random!
- Option A: *Learn* the representation from data.

Feature Learning



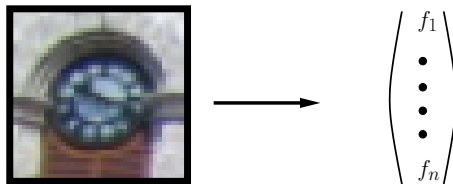
- How do we get good features?
- Option B: Engineer them. SIFT, HOG, LBP, etc.
- But: Natural Images are not Random!
- Option A: *Learn* the representation from data.

Feature Learning



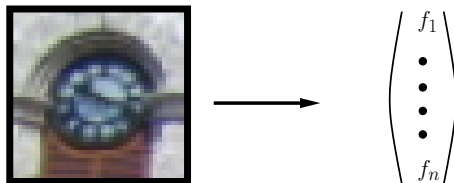
- How do we get good features?
- Option B: Engineer them. SIFT, HOG, LBP, etc.
- But: Natural Images are not Random!
- Option A: *Learn* the representation from data.

Feature Learning



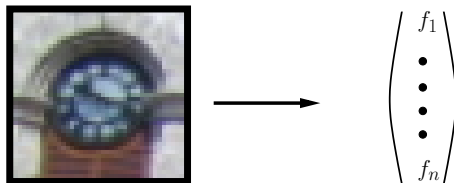
- How do we get good features?
- Option B: Engineer them. SIFT, HOG, LBP, etc.
- But: Natural Images are not Random!
- Option A: *Learn* the representation from data.

Feature Learning



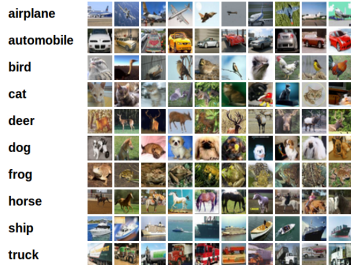
- Feature Learning, Dictionary Learning, Receptive Field Learning, Sparse Coding, etc.
- Helps overcome tedious engineering.
- Helps adapt to domains (including it's own representations!)
- Biologically consistent.
- Brings us much closer to *end-to-end learning* of vision systems.

Feature Learning

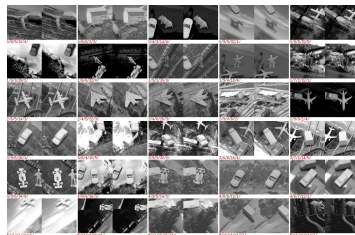


- Feature Learning, Dictionary Learning, Receptive Field Learning, Sparse Coding, etc.
- Helps overcome tedious engineering.
- Helps adapt to domains (including it's own representations!)
- Biologically consistent.
- Brings us much closer to *end-to-end learning* of vision systems.

Feature Learning Works



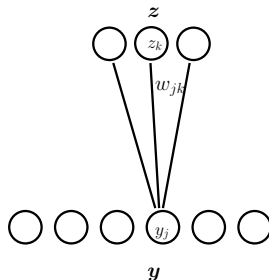
(CIFAR)



(NORB)

- More importantly... it works well!
- See, eg., (Coates, et al., 2011)

Feature Learning

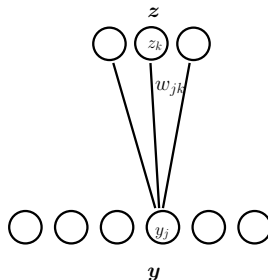


Feature Learning

- Encode patch y using latent variables z .
- Learn weights W from training data of image patches.

Feature Learning works well for recognition, so...

Feature Learning



Feature Learning

- Encode patch y using latent variables z .
- Learn weights W from training data of image patches.

Feature Learning works well for recognition, so...

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

- Applications
- Conclusions

Can we do more with Feature Learning than recognize *things*?

- Good features work well for object recognition.
- But brains can do much more than recognize objects.
- A large number of vision tasks goes beyond object recognition.
- In surprisingly many vision tasks, the relationship *between* images carries the relevant information

Correspondences in Computer Vision

- **Correspondence** is one of the most ubiquitous problems in Computer Vision.

Some correspondence tasks in Vision

- **Tracking**
- Stereo
- Optical Flow
- Invariant Recognition
- Odometry
- Action Recognition
- Contours, Within-image structure

Correspondences in Computer Vision

- **Correspondence** is one of the most ubiquitous problems in Computer Vision.

Some correspondence tasks in Vision

- Tracking
- **Stereo**
- Optical Flow
- Invariant Recognition
- Odometry
- Action Recognition
- Contours, Within-image structure

Correspondences in Computer Vision

- **Correspondence** is one of the most ubiquitous problems in Computer Vision.

Some correspondence tasks in Vision

- Tracking
- Stereo
- **Optical Flow**
- Invariant Recognition
- Odometry
- Action Recognition
- Contours, Within-image structure

Correspondences in Computer Vision

- **Correspondence** is one of the most ubiquitous problems in Computer Vision.

Some correspondence tasks in Vision

- Tracking
- Stereo
- Optical Flow
- **Invariant Recognition**
- Odometry
- Action Recognition
- Contours, Within-image structure

Correspondences in Computer Vision

- **Correspondence** is one of the most ubiquitous problems in Computer Vision.

Some correspondence tasks in Vision

- Tracking
- Stereo
- Optical Flow
- Invariant Recognition
- **Odometry**
- Action Recognition
- Contours, Within-image structure

Correspondences in Computer Vision

- **Correspondence** is one of the most ubiquitous problems in Computer Vision.

Some correspondence tasks in Vision

- Tracking
- Stereo
- Optical Flow
- Invariant Recognition
- Odometry
- **Action Recognition**
- Contours, Within-image structure

Correspondences in Computer Vision

- **Correspondence** is one of the most ubiquitous problems in Computer Vision.

Some correspondence tasks in Vision

- Tracking
- Stereo
- Optical Flow
- Invariant Recognition
- Odometry
- Action Recognition
- **Contours, Within-image structure**

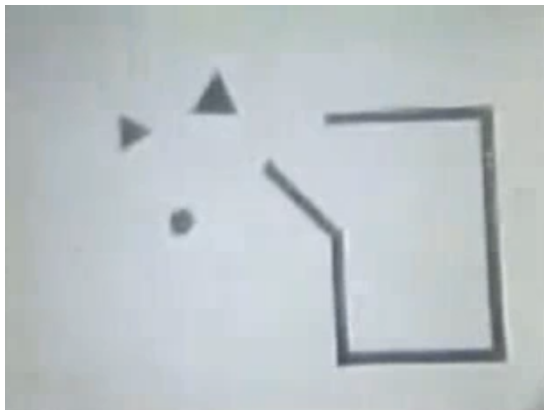
Correspondences in Computer Vision

- **Correspondence** is one of the most ubiquitous problems in Computer Vision.

Some correspondence tasks in Vision

- Tracking
- Stereo
- Optical Flow
- Invariant Recognition
- Odometry
- **Action Recognition**
- Contours, Within-image structure

Heider and Simmel



- See, for example, Heider and Simmel's 1944 video.
- Any single frame a bunch of geometric figures.
- The relations between frames reveal the content.

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

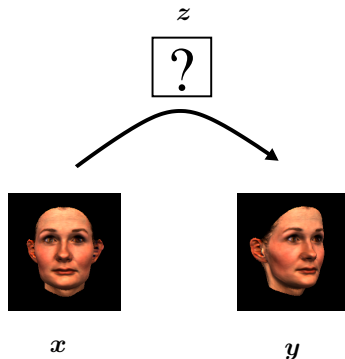
- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

- Applications
- Conclusions

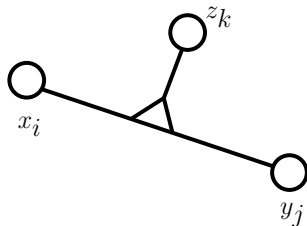
Learning features to model correspondences

- If *correspondences* matter in vision, **can we learn them?**



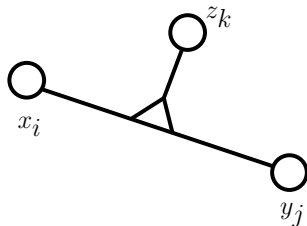
Learning features to model correspondences

- Yes, we can –
- if we allow latent variables to act like *gates*, that dynamically change the connections between fellow variables.



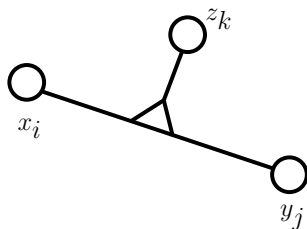
Learning features to model correspondences

- This amounts to letting variables multiply connections between other variables.
- And it is equivalent to having *three-way multiplicative interactions*.



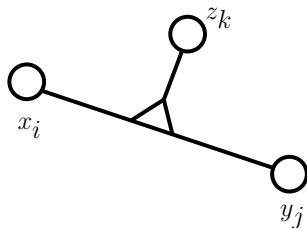
Learning features to model correspondences

- Learning and inference in the presence of these gating variables (slightly) different from learning models without.
- Can set things up, such that learning almost unchanged. But the *meaning* of latent variables entirely different.



Learning features to model correspondences

- Multiplicative interactions allow hidden variables to *blend in a whole “sub”-network*.
- This leads to qualitatively quite different behaviour than common bi-partite feature learning.



Multiplicative interactions

Brief history of multiplication

- “Mapping units” (Hinton; 1981), “dynamic mappings” (v.d. Malsburg; 1981)
- Binocular+Motion Energy models (Adelson, Bergen; 1985), (Ozhawa, DeAngelis, Freeman; 1990), (Fleet et al., 1994)
- Higher-order neural nets, “Sigma-Pi-units”
- Bi-linear models (Tenenbaum, Freeman; 2000), (Ohlshausen; 1994), (Grimes, Rao; 2005)
- Subspace SOM (Kohonen, 1996)
- ISA, topographic ICA (Hyvarinen, Hoyer; 2000), (Karklin, Lewicki; 2003): Higher-order within image structure
- (2006 –) GBM, mcRBM, RAE, convISA, applications...

Multiplicative interactions

Brief history of multiplication

- “Mapping units” (Hinton; 1981), “dynamic mappings” (v.d. Malsburg; 1981)
- Binocular+Motion Energy models (Adelson, Bergen; 1985), (Ozhawa, DeAngelis, Freeman; 1990), (Fleet et al., 1994)
- Higher-order neural nets, “Sigma-Pi-units”
- Bi-linear models (Tenenbaum, Freeman; 2000), (Ohlshausen; 1994), (Grimes, Rao; 2005)
- Subspace SOM (Kohonen, 1996)
- ISA, topographic ICA (Hyvarinen, Hoyer; 2000), (Karklin, Lewicki; 2003): Higher-order within image structure
- (2006 –) GBM, mcRBM, RAE, convISA, applications...

Brief history of multiplication

- “Mapping units” (Hinton; 1981), “dynamic mappings” (v.d. Malsburg; 1981)
- Binocular+Motion Energy models (Adelson, Bergen; 1985), (Ozhawa, DeAngelis, Freeman; 1990), (Fleet et al., 1994)
- **Higher-order neural nets, “Sigma-Pi-units”**
- Bi-linear models (Tenenbaum, Freeman; 2000), (Ohlshausen; 1994), (Grimes, Rao; 2005)
- Subspace SOM (Kohonen, 1996)
- ISA, topographic ICA (Hyvarinen, Hoyer; 2000), (Karklin, Lewicki; 2003): Higher-order within image structure
- (2006 –) GBM, mcRBM, RAE, convISA, applications...

Multiplicative interactions

Brief history of multiplication

- “Mapping units” (Hinton; 1981), “dynamic mappings” (v.d. Malsburg; 1981)
- Binocular+Motion Energy models (Adelson, Bergen; 1985), (Ozhawa, DeAngelis, Freeman; 1990), (Fleet et al., 1994)
- Higher-order neural nets, “Sigma-Pi-units”
- **Bi-linear models (Tenenbaum, Freeman; 2000), (Ohlshausen; 1994), (Grimes, Rao; 2005)**
- Subspace SOM (Kohonen, 1996)
- ISA, topographic ICA (Hyvarinen, Hoyer; 2000), (Karklin, Lewicki; 2003): Higher-order within image structure
- (2006 –) GBM, mcRBM, RAE, convISA, applications...

Multiplicative interactions

Brief history of multiplication

- “Mapping units” (Hinton; 1981), “dynamic mappings” (v.d. Malsburg; 1981)
- Binocular+Motion Energy models (Adelson, Bergen; 1985), (Ozhawa, DeAngelis, Freeman; 1990), (Fleet et al., 1994)
- Higher-order neural nets, “Sigma-Pi-units”
- Bi-linear models (Tenenbaum, Freeman; 2000), (Ohlshausen; 1994), (Grimes, Rao; 2005)
- **Subspace SOM (Kohonen, 1996)**
- ISA, topographic ICA (Hyvarinen, Hoyer; 2000), (Karklin, Lewicki; 2003): Higher-order within image structure
- (2006 –) GBM, mcRBM, RAE, convISA, applications...

Multiplicative interactions

Brief history of multiplication

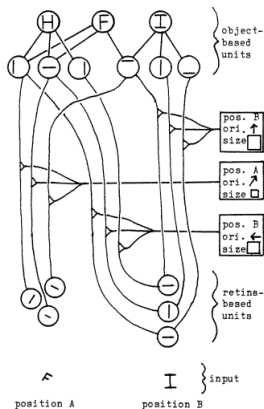
- “Mapping units” (Hinton; 1981), “dynamic mappings” (v.d. Malsburg; 1981)
- Binocular+Motion Energy models (Adelson, Bergen; 1985), (Ozhawa, DeAngelis, Freeman; 1990), (Fleet et al., 1994)
- Higher-order neural nets, “Sigma-Pi-units”
- Bi-linear models (Tenenbaum, Freeman; 2000), (Ohlshausen; 1994), (Grimes, Rao; 2005)
- Subspace SOM (Kohonen, 1996)
- ISA, topographic ICA (Hyvarinen, Hoyer; 2000), (Karklin, Lewicki; 2003): Higher-order within image structure
- (2006 –) GBM, mcRBM, RAE, convISA, applications...

Multiplicative interactions

Brief history of multiplication

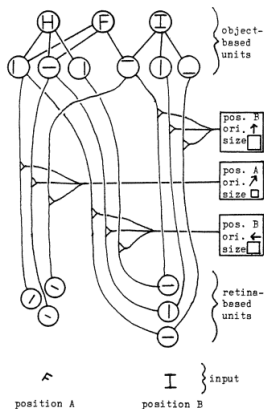
- “Mapping units” (Hinton; 1981), “dynamic mappings” (v.d. Malsburg; 1981)
- Binocular+Motion Energy models (Adelson, Bergen; 1985), (Ozhawa, DeAngelis, Freeman; 1990), (Fleet et al., 1994)
- Higher-order neural nets, “Sigma-Pi-units”
- Bi-linear models (Tenenbaum, Freeman; 2000), (Ohlshausen; 1994), (Grimes, Rao; 2005)
- Subspace SOM (Kohonen, 1996)
- ISA, topographic ICA (Hyvarinen, Hoyer; 2000), (Karklin, Lewicki; 2003): Higher-order within image structure
- (2006 –) GBM, mcRBM, RAE, convISA, applications...

Mapping units 1981



(Hinton, 1981)

Mapping units 1981



(Hinton, 1981)

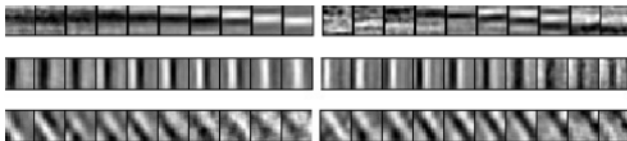
Example application: Action recognition



(Hollywood 2)

(Marszałek et al., 2009)

Example application: ISA applied to action recognition



- (Le, et al., 2011)

	KTH	Hollywood2	UCF	YouTube
until 2011	92.1	50.9	85.6	71.2
hierarchical ISA	93.9	53.3	86.5	75.8

Some examples: Bi-linear classification

- Let *labels act like gates*.

dataset/model:	SVMs		NNet	RBM	DEEP		GSM	
	SVMRBF	SVMPOL	NNet	DBN1	DBN3	SAA3	GSM	(unfact)
rectangles	2.15	2.15	7.16	4.71	2.60	2.41	0.83	(0.56)
rect-images	24.04	24.05	33.20	23.69	22.50	24.05	22.51	(23.17)
mnistplain	3.03	3.69	4.69	3.94	3.11	3.46	3.70	(3.98)
convexshapes	19.13	19.82	32.25	19.92	18.63	18.41	17.08	(21.03)
mnistbackrand	14.58	16.62	20.04	9.80	6.73	11.28	10.48	(11.89)
mnistbackimg	22.61	24.01	27.41	16.15	16.31	23.00	23.65	(22.07)
mnistrotbackimg	55.18	56.41	62.16	52.21	47.39	51.93	55.82	(55.16)
mnistrot	11.11	15.42	18.11	14.69	10.30	10.30	11.75	(16.15)

(Memisevic et al., 2010)

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

- Applications
- Conclusions

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

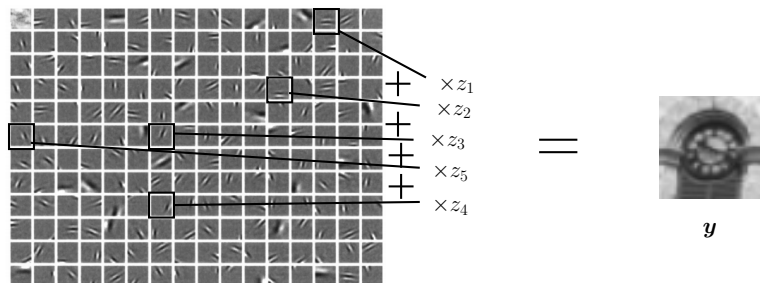
3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

- Applications
- Conclusions

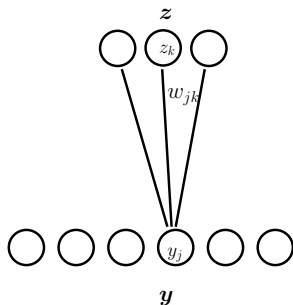
Sparse Coding Review



- Model an image-patch as the superposition of *basis functions*, or “*filters*”:

$$y = \sum_k W_{\cdot k} z_k, \quad y_j = \sum_k w_{jk} z_k$$

Sparse Coding Graphical Model

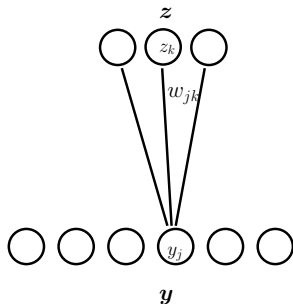


$$y_j^\alpha = \sum_k w_{jk} z_k^\alpha$$

Image synthesis

- Parameters w_{jk} connect pixels y_j with code components z_k
- Dimensionality of z can be smaller, larger, or same as y
- When the dimensionality is the same or larger, then z must be *constrained*, eg. by forcing it to be *sparse*.

Sparse Coding Graphical Model

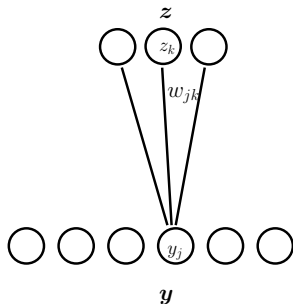


$$y_j^\alpha = \sum_k w_{jk} z_k^\alpha$$

Learning

- Given data-set $\mathbf{y}^1, \dots, \mathbf{y}^N$, adapt parameters W , inferring $\mathbf{z}^1, \dots, \mathbf{z}^N$ along the way.
- Unsupervised learning.

Sparse Coding Graphical Model



$$y_j^\alpha = \sum_k w_{jk} z_k^\alpha$$

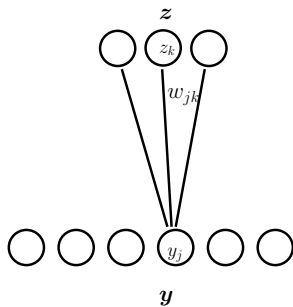
Learning

- For example

$$\min_{W, z^1, \dots, z^N} \frac{1}{N} \sum_{\alpha} (\|y^\alpha - \sum_k z_k^\alpha W_{.k}\|^2 + \lambda \sum_k |z_k^\alpha|)$$

- Alternating between W and all z^α .

Sparse Coding Graphical Model

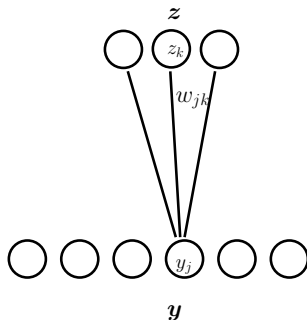


$$y_j^\alpha = \sum_k w_{jk} z_k^\alpha$$

Inference (“Analysis”)

- Given new image y , compute z .
- This is how we do recognition.

Sparse coding models

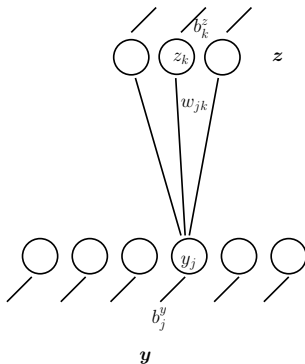


$$y_j = \sum_k w_{jk} z_k$$

Many Variants

- Probabilistic vs. Non-probabilistic
- Directed: define $p(\mathbf{y}|\mathbf{z})p(\mathbf{z})$ vs. undirected: define $p(\mathbf{y}, \mathbf{z})$
- Mixture models vs. factorial
- Symmetric vs. non-symmetric

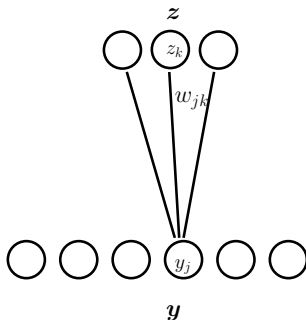
Sparse coding models



$$y_j = \sum_k w_{jk} z_k + b_j^y$$

- In practice: add bias terms.
- But we drop these for now to avoid clutter.

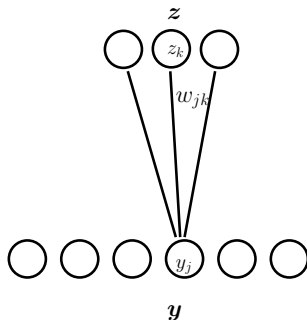
Sparse coding models



$$y_j = \sum_k w_{jk} z_k$$

- Some sparse coding models make inference easy:

Sparse coding models



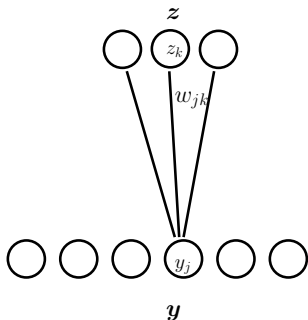
$$p(y_j|z) = \text{sigmoid}\left(\sum_k w_{jk} z_k\right)$$

$$p(z_k|y) = \text{sigmoid}\left(\sum_j w_{jk} y_j\right)$$

Restricted Boltzmann machines (RBM)

- $p(\mathbf{y}, \mathbf{z}) = \frac{1}{Z} \exp\left(\sum_{jk} w_{jk} y_j z_k\right)$
- Contrastive Divergence learning (Hebbian-style learning)
- Inference: $p(z_k|\mathbf{y}) = \text{sigmoid}\left(\sum_j w_{jk} y_j\right)$
- Further advantage: Allows for stacking (deep learning).

Sparse coding models



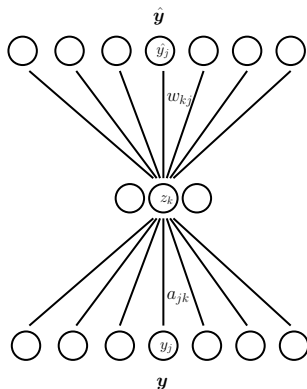
$$p(y_j|z) = \text{sigmoid}\left(\sum_k w_{jk} z_k\right)$$

$$p(z_k|y) = \text{sigmoid}\left(\sum_j w_{jk} y_j\right)$$

Restricted Boltzmann machines (RBM)

- $p(y, z) = \frac{1}{Z} \exp\left(\sum_{jk} w_{jk} y_j z_k\right)$
- Contrastive Divergence learning (Hebbian-style learning)
- Inference: $p(z_k|y) = \text{sigmoid}\left(\sum_j w_{jk} y_j\right)$
- Further advantage: Allows for stacking (deep learning).

Sparse coding models



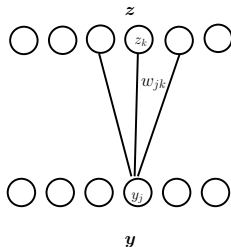
$$z_k = \text{sigmoid}\left(\sum_j a_{jk} y_j\right)$$

$$y_j = \sum_k w_{jk} z_k$$

(De-noising) autoencoders

- Add **inference parameters** A , and set $z = \text{sigmoid}(A\mathbf{y})$
- Learning: $\min_{W,A} \sum_{\alpha} \|\mathbf{y}^{\alpha} - W \text{sigmoid}(A\mathbf{y}^{\alpha})\|^2$
- *Corrupt inputs during training* (Vincent, 2008)

Sparse coding models



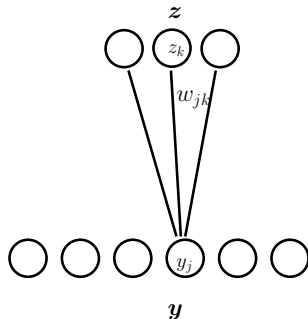
$$y_j = \sum_k w_{jk} z_k$$

ICA

- Learning: Make responses sparse while keeping filters sensible

$$\begin{aligned} \min_W & \|W^T \mathbf{y}\|_1 \\ \text{s.t.} \quad & W^T W = I \end{aligned}$$

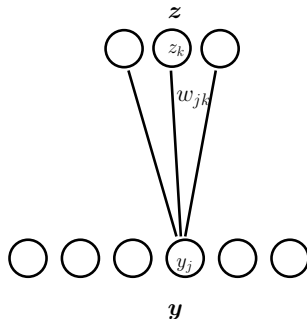
Sparse coding summary



Feature learning summary

- There are a lot of approaches to sparse coding.
- All methods yield Gabor filters when trained on natural images.
- All based on the same **rationale**:
 - Tease apart the hidden causes of variability in the data.

Sparse coding summary



Feature learning summary

- There are a lot of approaches to sparse coding.
- All methods yield Gabor filters when trained on natural images.
- All based on the same **rationale**:
- Tease apart the hidden causes of variability in the data.

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

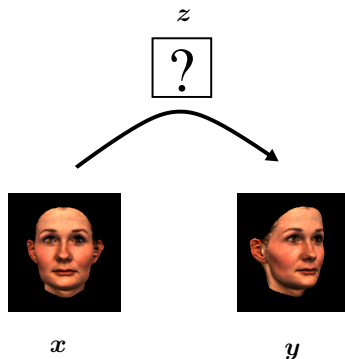
3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

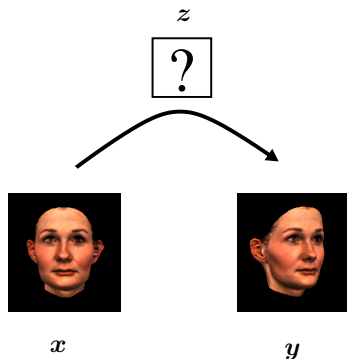
- Applications
- Conclusions

Sparse coding of images pairs?



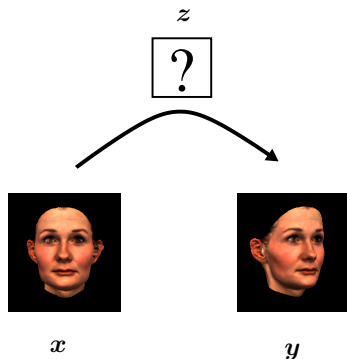
- How to extend sparse coding to model relations?
- Sparse coding on the *concatenation*?

Sparse coding of images pairs?



- How to extend sparse coding to model relations?
- Sparse coding on the *concatenation*?

Sparse coding of images pairs?

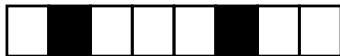


- How to extend sparse coding to model relations?
- Sparse coding on the *concatenation*?

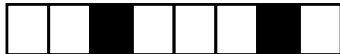
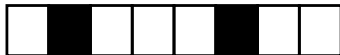
Sparse coding on the concatenation ?

- A case study: Shifted one-d images (binary).
- Suppose images are random and can change in **one of three ways**:

Example Image 1:

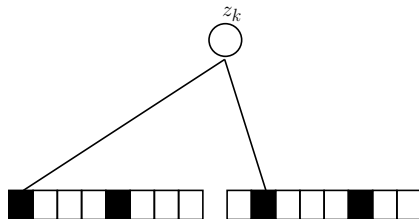


Possible Image 2:



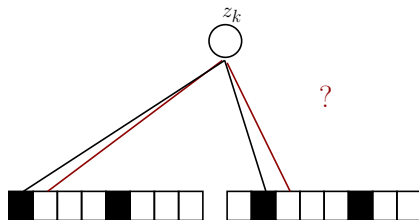
Sparse coding on the concatenation ?

- A hidden variable that collects evidence for a shift to the right.



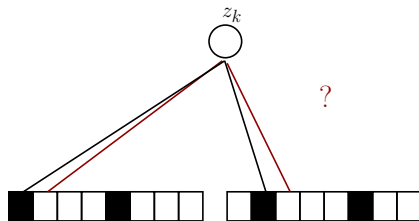
Sparse coding on the concatenation ?

- Can this variable *pool* over more than one pixel to fend off noise?



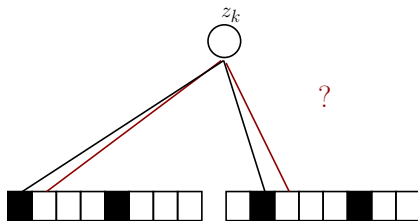
Sparse coding on the concatenation ?

- No, because now the hidden unit would get equally happy if it sees the non-shift (second pixel from the left).
- Hidden variables act like OR-gates, that accumulate evidence.



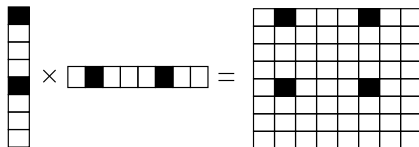
Sparse coding on the concatenation ?

- Both, noise or a different input pattern give rise to *some* evidence for *any* transformation.
- Hidden responses depend on the *content* of the input.



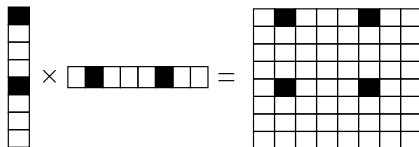
Cross-products

- A much better idea:
- Take $\text{outer}(x, y)$ to encode the relations:
- Use products not sums.



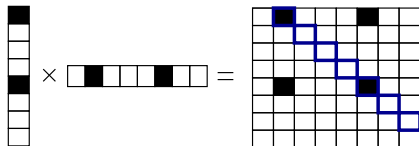
Cross-products

- Every component L_{ij} of this matrix will constitute evidence for exactly *one* type of shift.
- The components L_{ij} act like AND-gates, that can detect *coincidences* (eg. Zetsche et al., 2003, 2005).



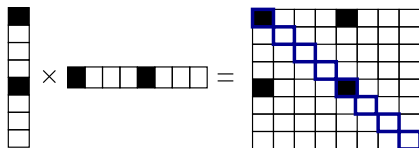
Cross-products

- Every component L_{ij} of this matrix will constitute evidence for exactly *one* type of shift.
- The components L_{ij} act like AND-gates, that can detect *coincidences* (eg. Zetsche et al., 2003, 2005).



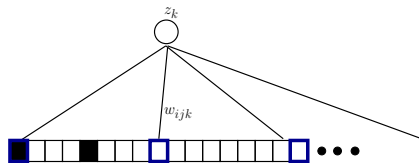
Cross-products

- Every component L_{ij} of this matrix will constitute evidence for exactly *one* type of shift.
- The components L_{ij} act like AND-gates, that can detect *coincidences* (eg. Zetsche et al., 2003, 2005).



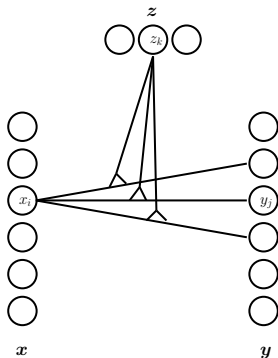
Cross-products

- Unroll $L : \text{vec}(\text{outer}(x, y))$
- Let hidden variables encode this vector, so they can learn to accumulate evidence for each type of transformation:



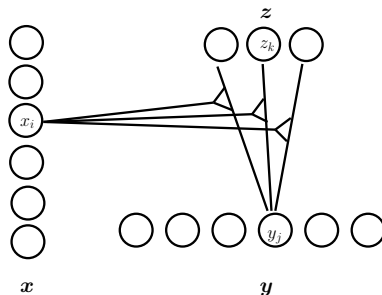
- Parameters turn into a three-way “tensor” with components w_{ijk} .
- Similarly for real valued inputs.
- (Tenenbaum, Freeman; 2000), (Grimes, Rao; 2005), (Ohlshausen; 2007), (Memisevic, Hinton; 2007)

An alternative visualization



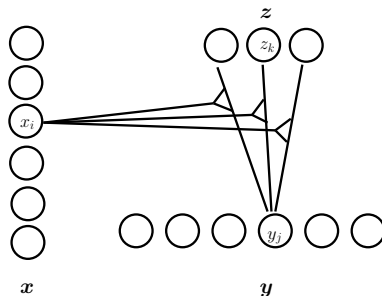
- This is a **factorial mixture of warps**:
- Each hidden variable can blend in a slice $W_{..k}$ of the parameter tensor.
- Each slice does linear regression in “pixel space”

Yet another visualization



- A different view:
- **“Conditional” sparse coding:**
- A sparse coding model, whose parameters are modulated case-by-case.

The kinds of transformation this captures



- Hidden variables can “push pixels around”.
- Every linear transformation in pixel-space is a *warp*.
- “Structural transformations”.
- To model color changes, “orthogonalize” intensity encoding.

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

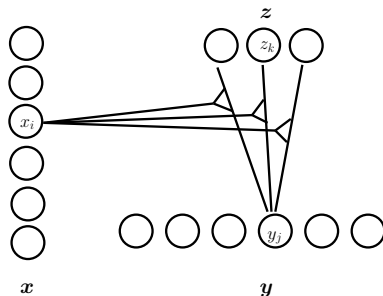
3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

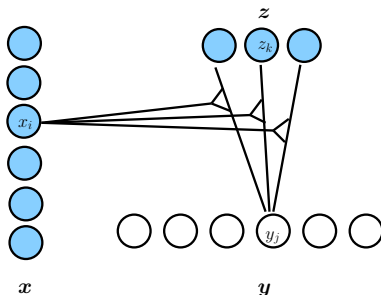
4 Applications

- Applications
- Conclusions

Inference

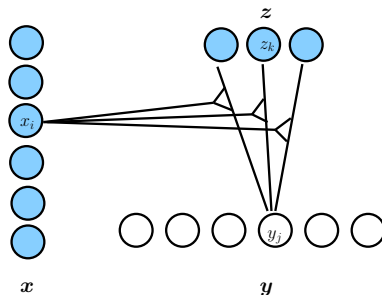


- Given any two sets of variables, it is easy to infer the third.
- As a result, inference is basically the same as in any standard sparse coding model.
- (Graph is tri-partite, sparse coding bi-partite.)



Inferring y (given x and z)

- Recall that, for vanilla sparse coding: $y_j = \sum_k w_{jk} z_k$
- Conditional sparse coding turns the weights into a function of x .

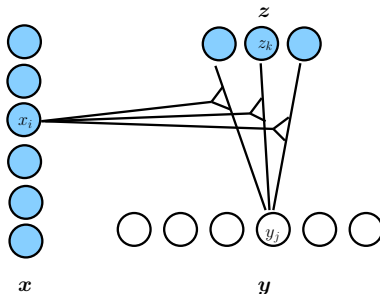


Inferring y (given x and z)

- If we make that function a linear one, $w_{jk}(x) = \sum_i w_{ijk}x_i$, we get

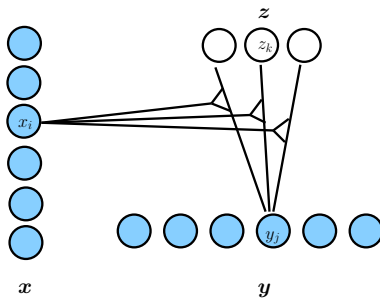
$$y_j = \sum_k w_{jk}z_k = \sum_k \left(\sum_i w_{ijk}x_i \right) z_k = \sum_{ik} w_{ijk}x_i z_k$$

Inference



Inferring y (given x and z)

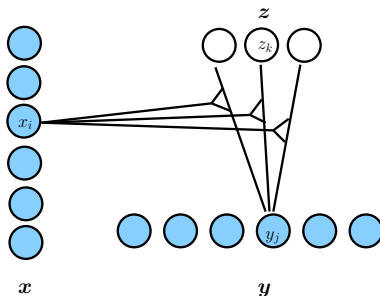
- The *meaning* differs from vanilla sparse coding: y now represents “ x transformed according to z ”.



Inferring z (given x and y)

- Inference for z analogous. Given two images, x and y , infer the transformation z by modulating parameters linearly.

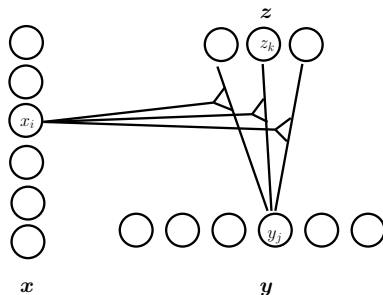
$$z_k = \sum_j w_{jk} y_j = \sum_k \left(\sum_i w_{ijk} x_i \right) y_j = \sum_{ij} w_{ijk} x_i y_j$$



Inferring z (given x and y)

- Meaning of z : The *transformation* that takes x to y (or vice versa).

Inference



- Inference can mean various other things in addition.
- For example, given x and y , **how likely are these to come together?**
- More on this type of inference later.

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

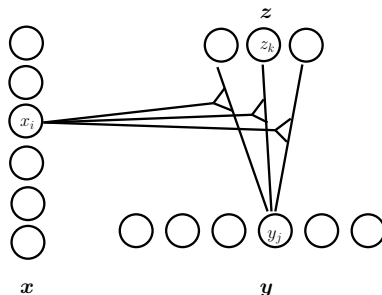
- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

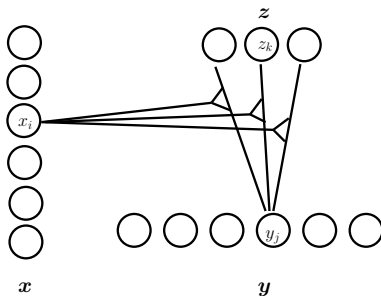
4 Applications

- Applications
- Conclusions



- Training data: (x^α, y^α) -pairs – the points we want to relate.
- The simplest way to train this model is **conditionally**:
- Learning is “sparse coding with case-dependent weights”.
- In other words: For *any given training-case* (x^α, y^α) , we have a standard sparse coding model whose parameters are modulated by the inputs.

Conditional parameter updates

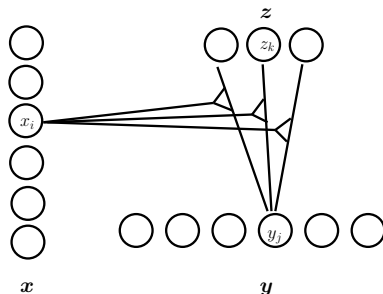


- Thus, the cost that data-case (x^α, y^α) contributes is:

$$\sum_j (y_j^\alpha - \sum_{ik} w_{ijk} x_i^\alpha z_k^\alpha)^2$$

- Differentiating with respect to w_{ijk} just like before.
- Model is still *linear* wrt. parameters.

Example: Gated Boltzmann machine

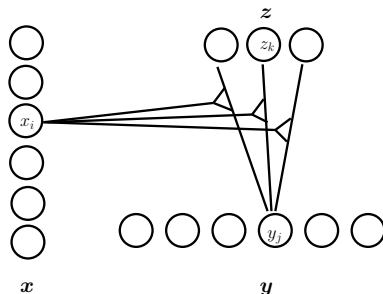


- For a restricted Boltzmann machine, this amounts to changing the energy function into a *three-way energy*:

$$E(\mathbf{x}, \mathbf{y}, \mathbf{z}) = \sum_{ijk} w_{ijk} x_i y_j z_k$$

- Then $p(\mathbf{y}, \mathbf{z} | \mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp(E(\mathbf{x}, \mathbf{y}, \mathbf{z}))$,
 $Z(\mathbf{x}) = \sum_{\mathbf{y}, \mathbf{z}} \exp(E(\mathbf{x}, \mathbf{y}, \mathbf{z}))$

Example: Gated Boltzmann machine

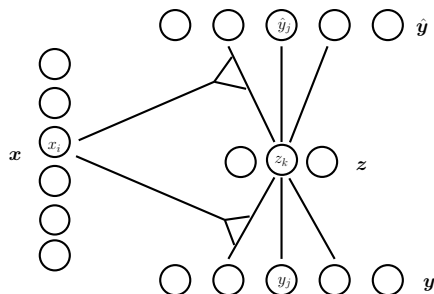


- For a restricted Boltzmann machine, this amounts to changing the energy function into a *three-way energy*:

$$E(\mathbf{x}, \mathbf{y}, \mathbf{z}) = \sum_{ijk} w_{ijk} x_i y_j z_k$$

- Then $p(\mathbf{y}, \mathbf{z} | \mathbf{x}) = \frac{1}{Z(\mathbf{x})} \exp(E(\mathbf{x}, \mathbf{y}, \mathbf{z}))$,
 $Z(\mathbf{x}) = \sum_{\mathbf{y}, \mathbf{z}} \exp(E(\mathbf{x}, \mathbf{y}, \mathbf{z}))$

Example: Gated auto-encoder

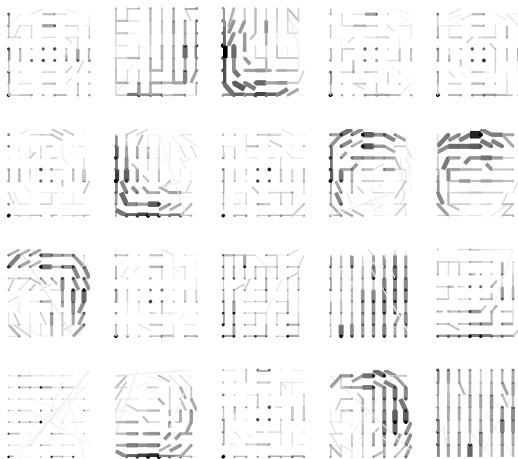


- Similar for autoencoders.
- Both, encoder and decoder weights turn into functions of x .
- Learning the same as in a standard auto-encoder modeling y .
- The model is still a DAG, so back-prop works unchanged.

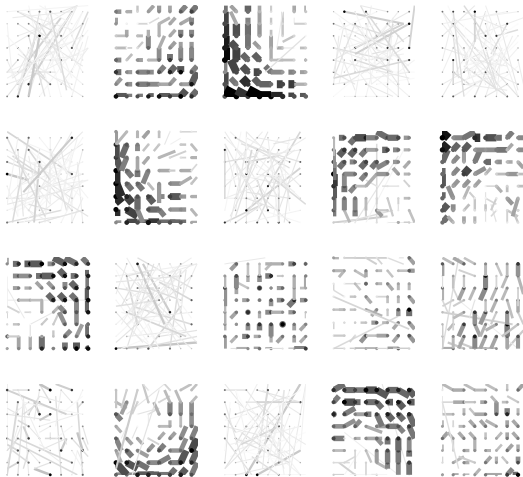
Other example

- (Ohlshausen et al.; 2007): Conditional, bi-linear sparse coding.

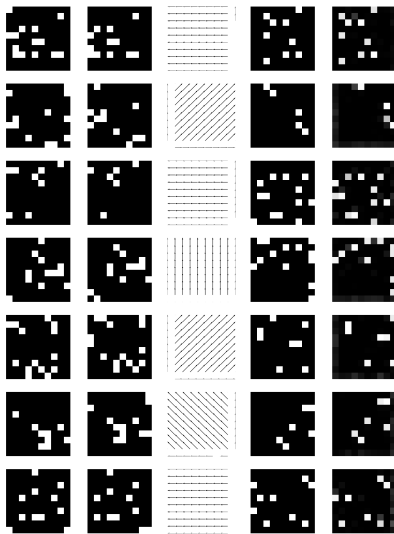
Toy example: Conditionally trained “Hidden flow-fields”



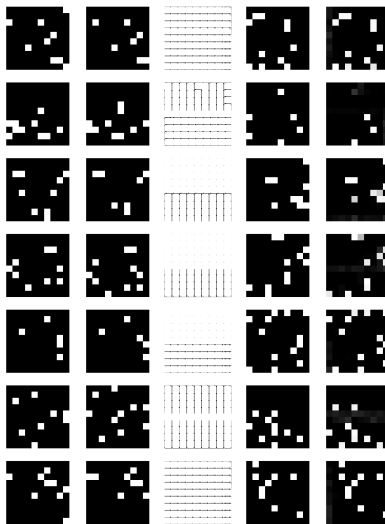
Toy example: Conditionally trained “Hidden flow-fields”, inhibitory connections



Toy example: Inferring the optical flow

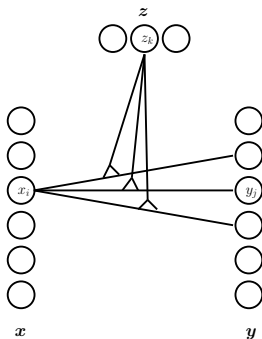


“Combinatorial flow”



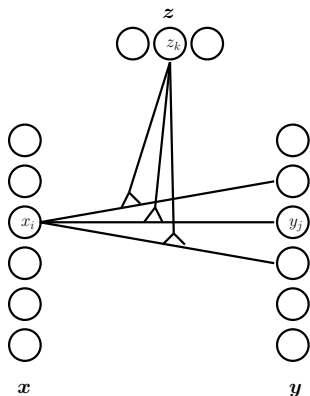
- Conditional training on videos is *predictive coding*:
- Model the next time frame, given the previous one.
- Inference then provides an encoding of the transformation.
- This is often a sensible strategy, but not always.

Joint training



- Conditional training makes it hard to answer questions like:
- “How likely are the given images transforms of one another?”
- To answer questions like these, we require a joint image model, $p(x, y|z)$, given mapping units.

Joint training



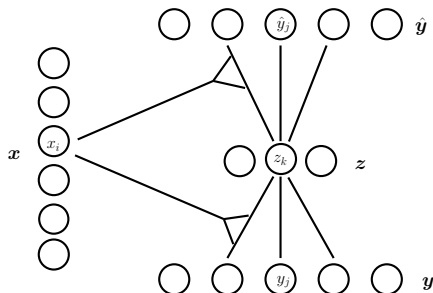
$$E(\mathbf{x}, \mathbf{y}, \mathbf{z}) = \sum_{ijk} w_{ijk} x_i y_j z_k$$

$$p(\mathbf{x}, \mathbf{y}, \mathbf{z}) = \frac{1}{Z} \exp(E(\mathbf{x}, \mathbf{y}, \mathbf{z}))$$

$$Z = \sum_{\mathbf{x}, \mathbf{y}, \mathbf{z}} \exp(E(\mathbf{x}, \mathbf{y}, \mathbf{z}))$$

- (Susskind et al., 2011): Three-way Gibbs sampling in a Gated Boltzmann Machine.
- Apply to matching tasks (more later).

Joint training

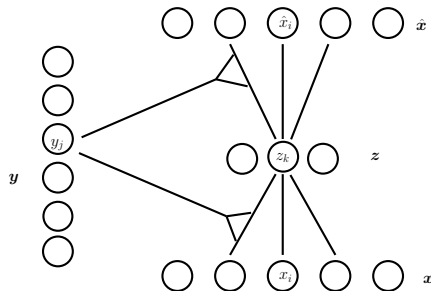


- For the auto-encoder, there exists a simple hack:
- Add up two conditional costs:

$$\sum_j (y_j^\alpha - \sum_{ik} w_{ijk} x_i^\alpha z_k^\alpha)^2 + \sum_i (x_i^\alpha - \sum_{jk} w_{ijk} y_j^\alpha z_k^\alpha)^2$$

- This forces parameters to be able to transform in both directions.

Joint training



- For the auto-encoder, there exists a simple hack:
- Add up two conditional costs:

$$\sum_j (y_j^\alpha - \sum_{ik} w_{ijk} x_i^\alpha z_k^\alpha)^2 + \sum_i (x_i^\alpha - \sum_{jk} w_{ijk} y_j^\alpha z_k^\alpha)^2$$

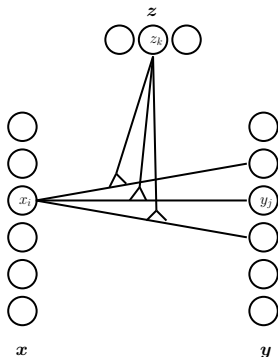
- This forces parameters to be able to transform in both directions.

Pool over products

Take-home message

To gather evidence for a transformation,
let each hidden unit **pool over products** of input-components.

Some references



- (Hinton; 1981), (v.d. Malsburg; 1981)
- (Grimes, Rao; 2005): Bi-linear sparse coding.
- (Tenenbaum, Freeman; 2000), (Grimes, Rao; 2005), (Ohlshausen; 2007), (Memisevic, Hinton; 2007), (Susskind, et al., 2011)
- (Zetzsche et al.; 2003, 2005)

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

- Applications
- Conclusions

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

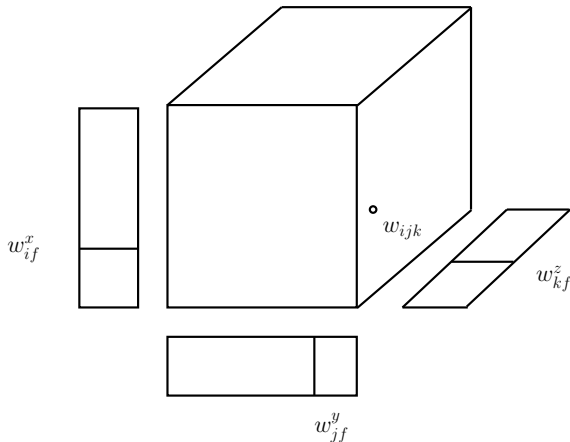
- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

- Applications
- Conclusions

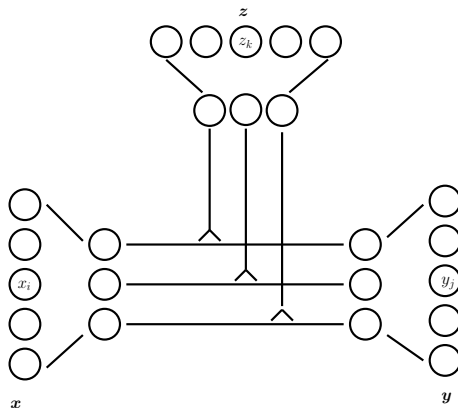
- The number of parameters is about $n \times n \times n$ (!)
- More, if we want sparse, overcomplete hidden.
- There is a simple, yet far-reaching, way to reduce that number.

Factorization



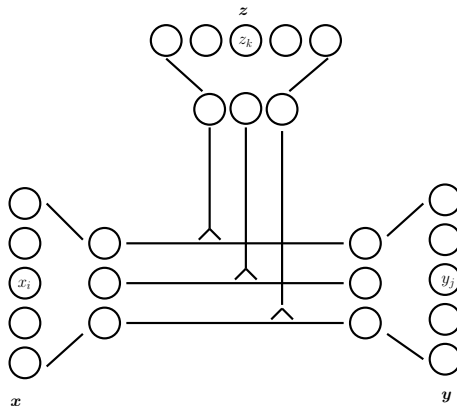
$$w_{ijk} = \sum_{ijk} \sum_f w_{if}^x w_{jf}^y w_{kf}^z$$

Factorization is *filter matching*



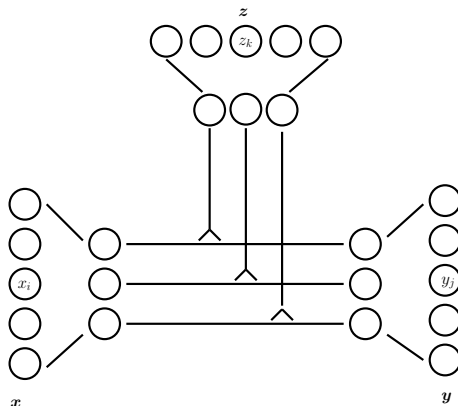
$$\begin{aligned} y_j &= \sum_{ik} w_{ijk} x_i z_k = \sum_{ik} \left(\sum_f w_{if}^x w_{jf}^y w_{kf}^z \right) x_i z_k \\ &= \sum_f w_{jf}^y \left(\sum_i w_{if}^x x_i \right) \left(\sum_k w_{kf}^z z_k \right) \end{aligned}$$

Factorization is *filter matching*



$$E = \sum_{ijk} \left(\sum_f w_{if}^x w_{jf}^y w_{kf}^z \right) x_i y_j z_k = \sum_f \left(\sum_i w_{if}^x x_i \right) \left(\sum_j w_{jf}^y y_j \right) \left(\sum_k w_{kf}^z z_k \right)$$

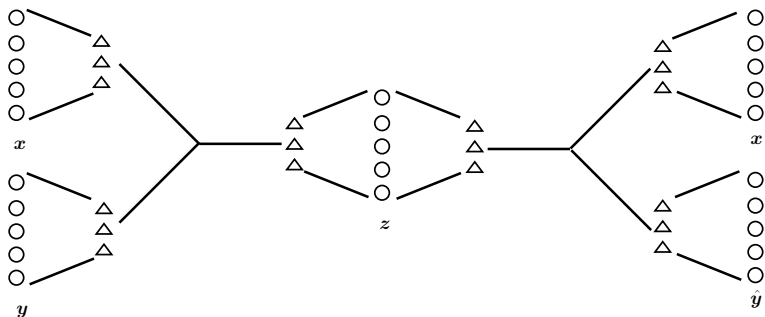
Factorized models



Factored Gated Boltzmann machines

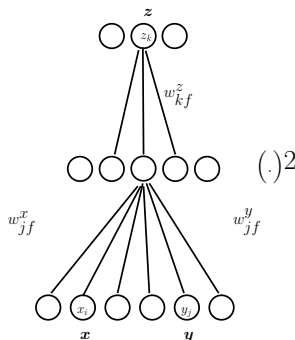
- Exponentiate and normalize energy (just like RBM).
- Learning and inference exactly like before.

Factorized models



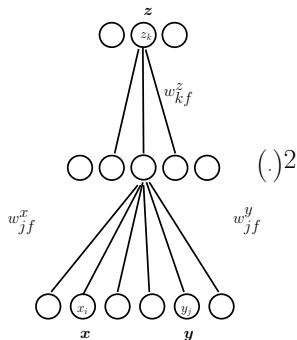
Factored Relational Autoencoders

- Again, everything like before. Back-propagate through the filters.
- Conditional learning trivial.
- Joint learning by adding two asymmetric objectives.



Independent Subspace Analysis

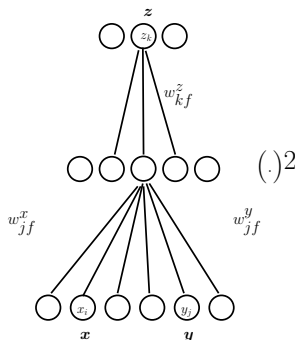
- “Subspace ICA”: Make z sparse, while keeping filters orthogonal.
- This is almost like ICA on concatenation. But features encode *sums-of-squares* of filter responses. (Hyvarinen, 2000), (Kohonen, 1996)



Independent Subspace Analysis

- Why is this doing the same?
- The activity that a hidden unit gets is:

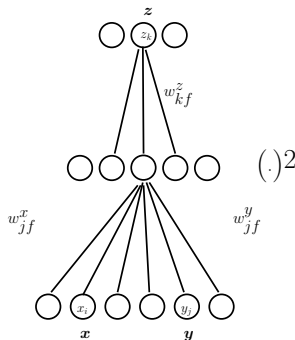
Factorized models



Independent Subspace Analysis

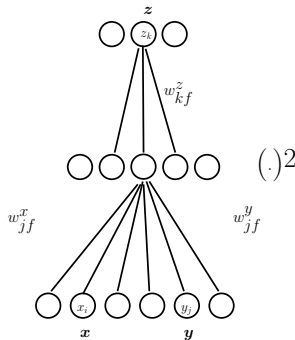
$$\begin{aligned} & \sum_f w_{kf}^z (w^x{}^T \mathbf{x} + w^y{}^T \mathbf{y})^2 \\ &= \sum_f w_{kf}^z (2(w_{\cdot f}^x{}^T \mathbf{x})(w_{\cdot f}^y{}^T \mathbf{y}) + (w_{\cdot f}^x{}^T \mathbf{x})^2 + (w_{\cdot f}^y{}^T \mathbf{y})^2) \end{aligned}$$

Factorized models



Independent Subspace Analysis

$$\begin{aligned} & \sum_f w_{kf}^z (w^{\mathbf{x}\top} \mathbf{x} + w^{\mathbf{y}\top} \mathbf{y})^2 \\ &= \sum_f w_{kf}^z (2(\mathbf{w}_{\cdot f}^{\mathbf{x}\top} \mathbf{x})(\mathbf{w}_{\cdot f}^{\mathbf{y}\top} \mathbf{y}) + (\mathbf{w}_{\cdot f}^{\mathbf{x}\top} \mathbf{x})^2 + (\mathbf{w}_{\cdot f}^{\mathbf{y}\top} \mathbf{y})^2) \end{aligned}$$



Independent Subspace Analysis

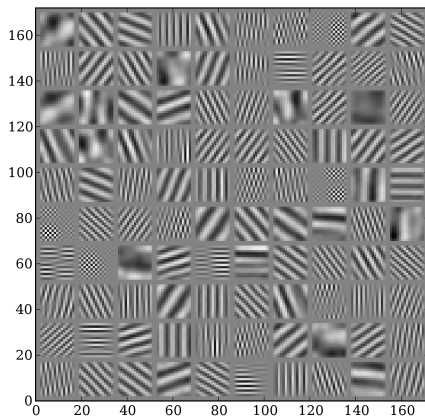
- Learning like ICA: Gradient-based, while enforcing $W^{y^T}W^y = I$ after every gradient step (eigen-decomposition).

Examples

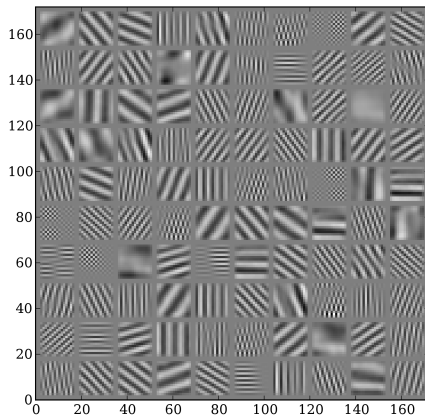


- Toy examples:
- There is no structure in these images.
- Only in *how they change*.

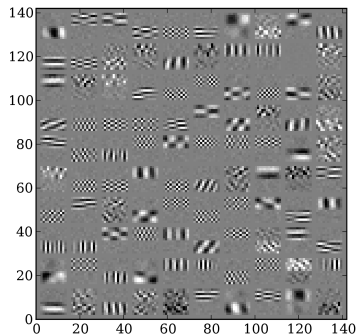
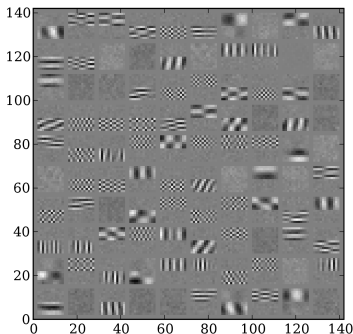
Learned filters w_{if}^x



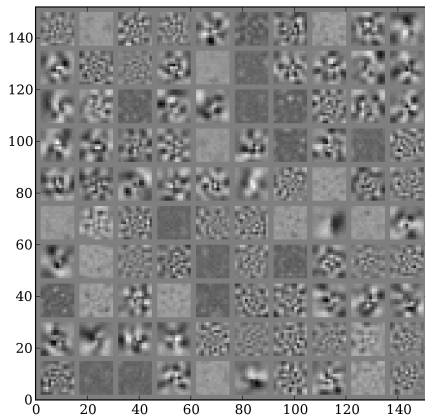
Learned filters w_{jf}^y



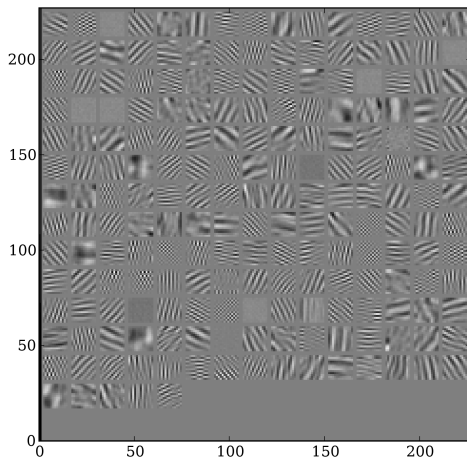
Filters learned from split-screen shifts



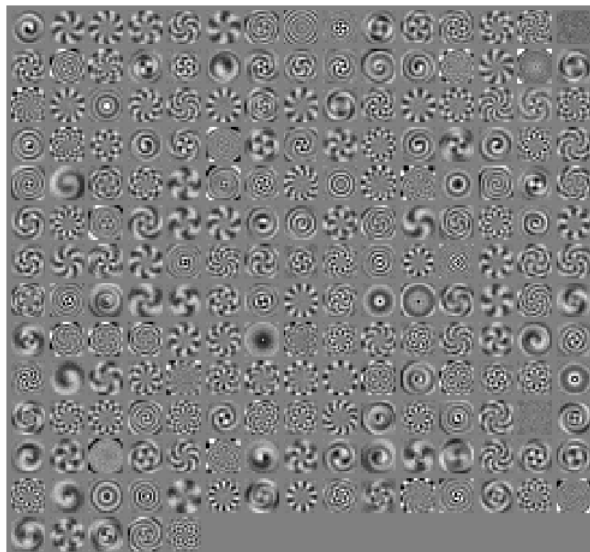
Affine filters



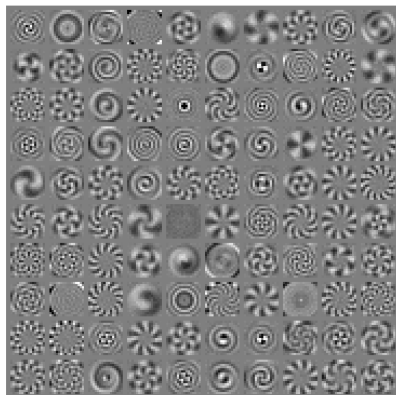
“Filtering”-filters



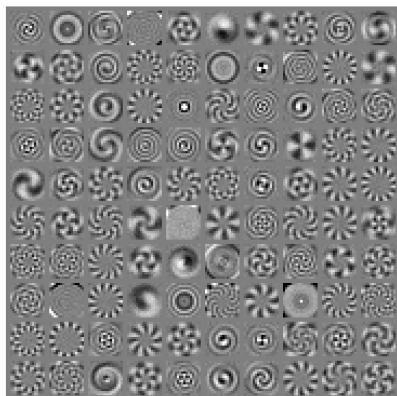
Rotation filters



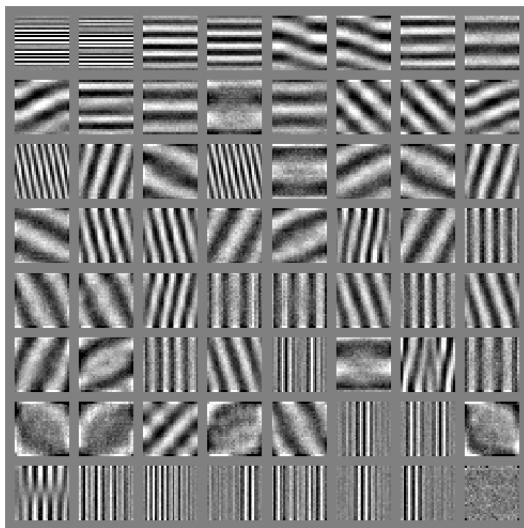
Rotation filters



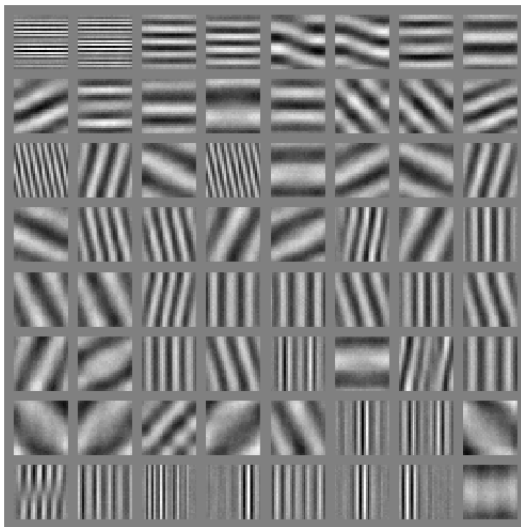
Rotation filters



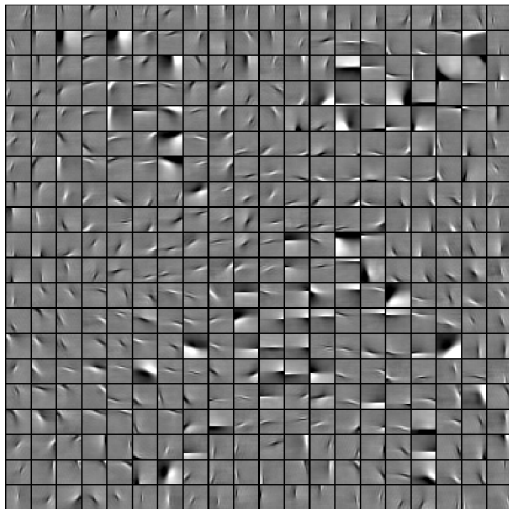
Filters learned by watching TV



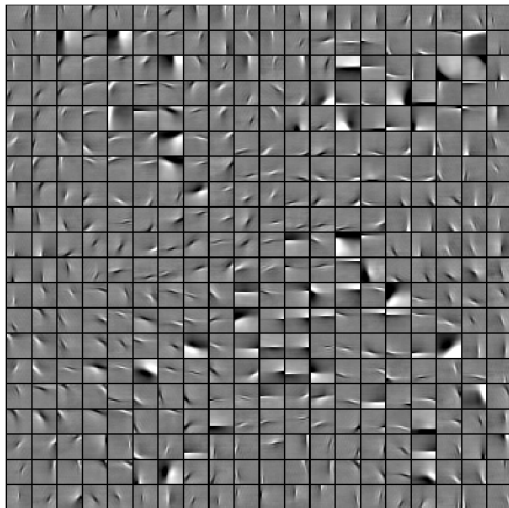
Filters learned by watching TV



“Bag-Of-Warps”



“Bag-Of-Warps”



1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

- Applications
- Conclusions

- (Bethge, 2007)
- Many interesting transformations, like translations, rotations, permutations, etc. can be expressed, approximately, as *orthogonal transformation* L in pixel space (up to edge effects):
 - Assume the input and output data are white: $C_y = C_x = I$, and there is no noise.
 - Then the covariance matrix on the outputs $y = Lx$ will be:

$$I = C_y = LC_x L^T = LL^T$$

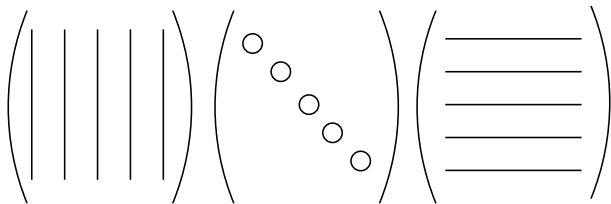
- Here is an interesting fact about orthogonal matrices:

- (Bethge, 2007)
- Many interesting transformations, like translations, rotations, permutations, etc. can be expressed, approximately, as *orthogonal transformation* L in pixel space (up to edge effects):
 - Assume the input and output data are white: $C_y = C_x = I$, and there is no noise.
 - Then the covariance matrix on the outputs $y = Lx$ will be:
$$I = C_y = LC_xL^T = LL^T$$
- Here is an interesting fact about orthogonal matrices:

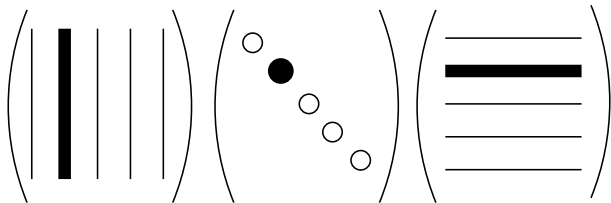
Eigen-decomposition of an orthogonal matrix

- An orthogonal matrix has an eigen-decomposition with *complex eigenvalues* that come in conjugate pairs (plus, maybe, some $+1$'s and -1 's).
- The eigenvalues have length 1, so they act as *rotations* in 2d.
- $\rightarrow$ Projecting onto the invariant sub-spaces of L allows us to decompose a transformation into a set of independent 2d-rotations.

$$L = UDU^T$$

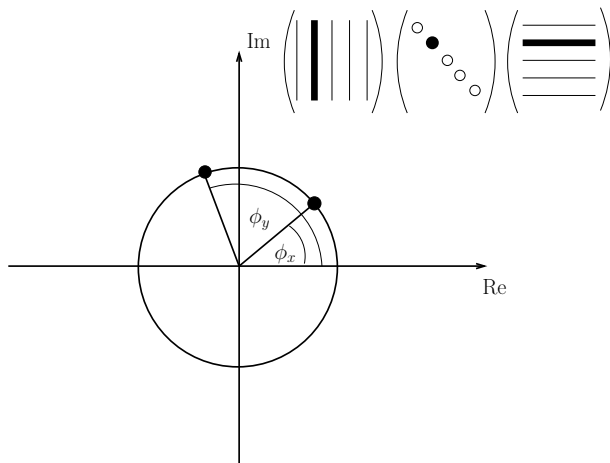


$$L = UDU^T$$

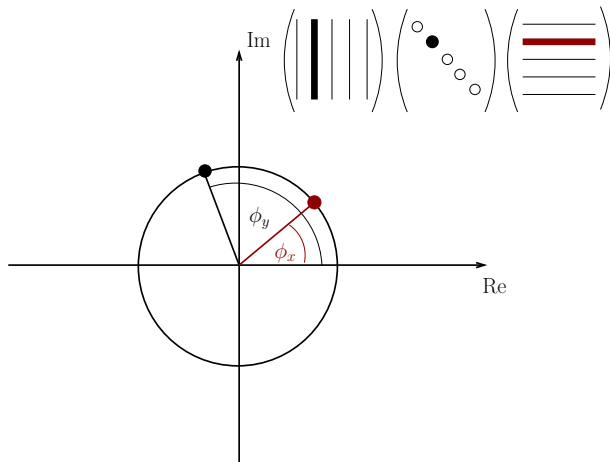


- (Each complex eigenvector may be thought of as two real feature vectors, one imaginary, one real. Like-wise the projections of data onto it: $(v^{R^T}x)$, $(v^{I^T}x)$, $(v^{R^T}y)$, $(v^{I^T}y)$)

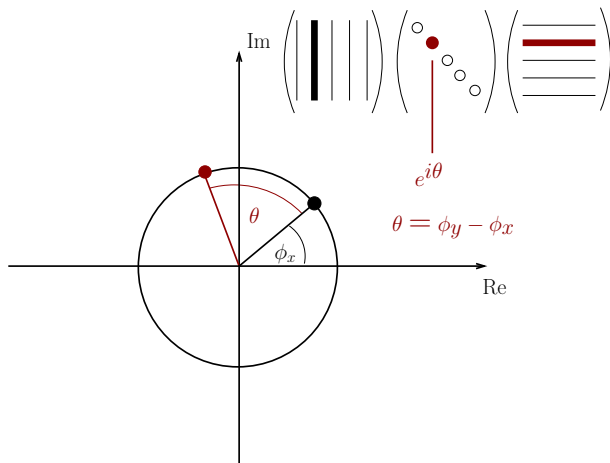
$$L = UDU^T$$



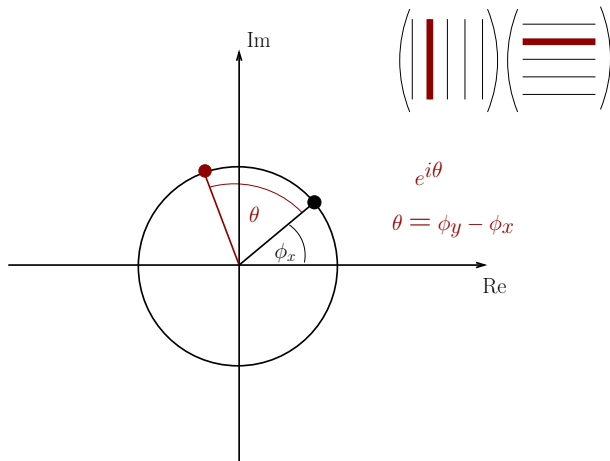
$$L = UDU^T$$



$$L = UDU^T$$



$$L = UDU^T$$



(Remark: We can absorb the eigen-value into the eigenvector.)

Example: Translation

Example: Translation and Fourier spectrum

- Example: The eigen-features of translation are Fourier-features (sine-/cosine-pairs).
- The rotations in the invariant subspaces correspond to phase-shifts of the Fourier-features.
- The norms of these features (this is the power spectrum of the signal) stay constant obviously, as is well known for translation.

Shared eigenspaces

- An important observation: The eigen-features for a different type of shift are ... the same (Fourier-features)!
- This means, we can, in principle, represent multiple shifts using a single appropriate feature representation.
- True for other types of transformation, too.

Example: Translation

Example: Translation and Fourier spectrum

- Example: The eigen-features of translation are Fourier-features (sine-/cosine-pairs).
- The rotations in the invariant subspaces correspond to phase-shifts of the Fourier-features.
- The norms of these features (this is the power spectrum of the signal) stay constant obviously, as is well known for translation.

Shared eigenspaces

- An important observation: The eigen-features for a different type of shift are ... the same (Fourier-features)!
- This means, we can, in principle, represent multiple shifts using a single appropriate feature representation.
- True for other types of transformation, too.

Extracting the rotations

- Given a shared eigen-representation, we can recover the rotation within each subspace as follows:
- Call $\theta = \phi_x - \phi_y$ the angle of rotation between the real and imaginary “eigen-features” $(\mathbf{v}^{R^T} \mathbf{x})$, $(\mathbf{v}^{I^T} \mathbf{x})$, $(\mathbf{v}^{R^T} \mathbf{y})$, $(\mathbf{v}^{I^T} \mathbf{y})$ of the given observation-pair.
- We have

$$\begin{aligned}\cos \theta &= \cos(\phi_y - \phi_x) \\ &= \cos \phi_y \cos \phi_x + \sin \phi_y \sin \phi_x \\ &= (\mathbf{v}^{R^T} \mathbf{y})(\mathbf{v}^{R^T} \mathbf{x}) + (\mathbf{v}^{I^T} \mathbf{y})(\mathbf{v}^{I^T} \mathbf{x})\end{aligned}$$

- Compute the *sum over products of filter responses*.

Extracting the rotations

- Given a shared eigen-representation, we can recover the rotation within each subspace as follows:
- Call $\theta = \phi_x - \phi_y$ the angle of rotation between the real and imaginary “eigen-features” $(\mathbf{v}^{R^T} \mathbf{x})$, $(\mathbf{v}^{I^T} \mathbf{x})$, $(\mathbf{v}^{R^T} \mathbf{y})$, $(\mathbf{v}^{I^T} \mathbf{y})$ of the given observation-pair.
- We have

$$\begin{aligned}\cos \theta &= \cos(\phi_y - \phi_x) \\ &= \cos \phi_y \cos \phi_x + \sin \phi_y \sin \phi_x \\ &= (\mathbf{v}^{R^T} \mathbf{y})(\mathbf{v}^{R^T} \mathbf{x}) + (\mathbf{v}^{I^T} \mathbf{y})(\mathbf{v}^{I^T} \mathbf{x})\end{aligned}$$

- Compute the *sum over products of filter responses*.

Extracting the rotations

- We can, similarly, build a shift-detector by absorbing the eigenvector into, say, the input-feature.
- Call $\theta = \phi_x - \phi_y$ the observed angle of rotation between the (now distinct input/output-)features $(\mathbf{v}_x^{R^T} \mathbf{x})$, $(\mathbf{v}_x^{I^T} \mathbf{x})$, $(\mathbf{v}_y^{R^T} \mathbf{y})$, $(\mathbf{v}_y^{I^T} \mathbf{y})$ of the given observation-pair.

$$\begin{aligned}\cos \theta &= \cos(\phi_y - \phi_x) \\ &= \cos \phi_y \cos \phi_x + \sin \phi_y \sin \phi_x \\ &= (\mathbf{v}_y^{R^T} \mathbf{y})(\mathbf{v}_x^{R^T} \mathbf{x}) + (\mathbf{v}_y^{I^T} \mathbf{y})(\mathbf{v}_x^{I^T} \mathbf{x})\end{aligned}$$

- This detector will peak when $\phi_y - \phi_x = 0$, that is, when the observation pair conforms with the preferred angle of this unit.

The cross-correlation model

- A hidden variable that computes a sum over products of filter responses can detect rotations θ in an eigen-subspace.
- It will have a peak response at a preferred angle of rotation in that eigen-space.
- Depending on the choice of eigen-basis, this hidden variable could detect shifts, rotations, local shifts, permutations, or other types of transformation.
- We now know how to *learn* these detectors from training data!

Relation to energy models

- Short-cut: Pool over **squared responses to the concatenation**:

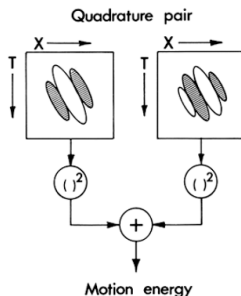
$$\begin{aligned}s^R(\mathbf{x}, \mathbf{y}) &:= \left(v^{R^T}[\mathbf{x}; \mathbf{y}]\right)^2 \\ &= \left(v_x^{R^T} \mathbf{x}\right)^2 + \left(v_y^{R^T} \mathbf{y}\right)^2 + 2(v_x^{R^T} \mathbf{x})(v_y^{R^T} \mathbf{y})\end{aligned}$$

- If filters and data roughly normalized, the variability of $s^R(\mathbf{x}, \mathbf{y})$ will be dominated by the cross-products. (Same argument as before: ISA-GBM).
- Computing the sum of squared responses of quadrature pairs has been known as:

The energy model

- (Adelson and Bergen, 1985): Motion
- (Ozhawa, DeAngelis, Freeman; 1990): Disparity
- Equivalence to cross-correlation: See, for example, (Fleet et al.; 1994).

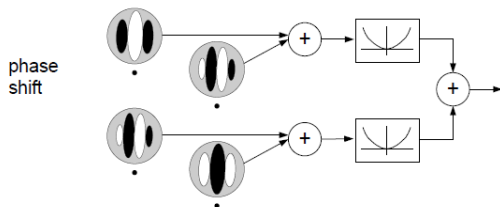
Energy Models



- (Adelson and Bergen, 1985).
- Project into a subspace that represents spatio-temporal patterns. Then compute the (squared) norm of each feature.
- Energy models were derived with respect to Fourier (Gabor) filters, where the sum of squares extracts the phase-independent power spectrum. (Therefore the term.)
- But the idea applies more generally!

Energy models and complex cells

- Energy models are also a standard model for the simple/complex cell dichotomy in the brain.



(Fleet et al.; 1994)

Cross-correlation and energy models

- There has been a lot of confusion surrounding energy models.
- Energy models can be applied both to a single static image and to image pairs.
- For the latter, energy models make obvious sense:

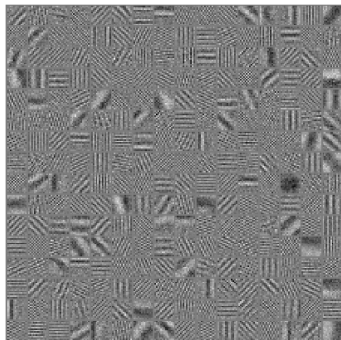
Encode transformations independently of image content.

- This exactly implements the sparse coding/ICA philosophy: Seek independent contributors to the variability in data.
- For static images, the situation is somewhat more fuzzy: “Localize edges... but not exactly.”

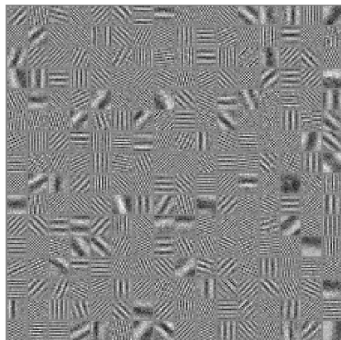
Extension to more than two frames

- It is straightforward to extend the model to more than two frames:
- Allow for cross-products between all pairs of responses.
- Alternatively: Concatenate, then sum the squared filter responses.

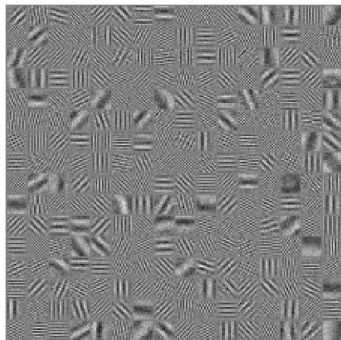
Extension to more than two frames



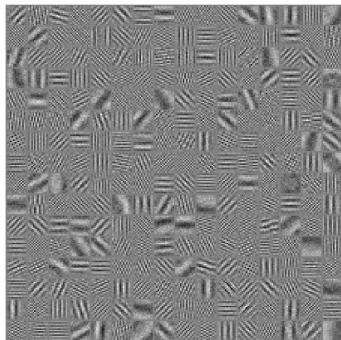
Extension to more than two frames



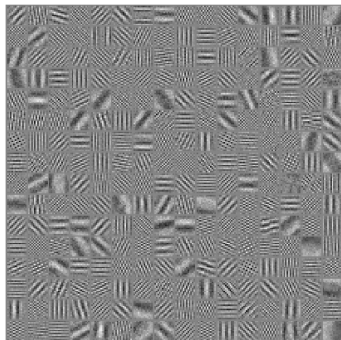
Extension to more than two frames



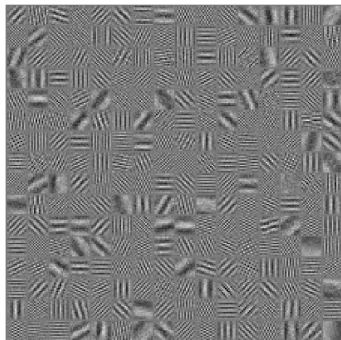
Extension to more than two frames



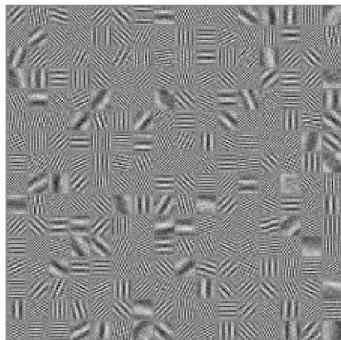
Extension to more than two frames



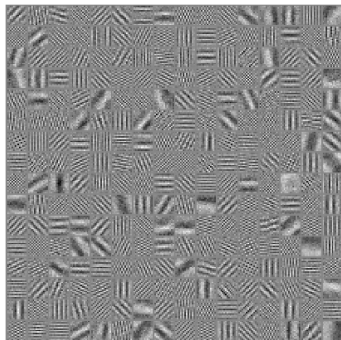
Extension to more than two frames



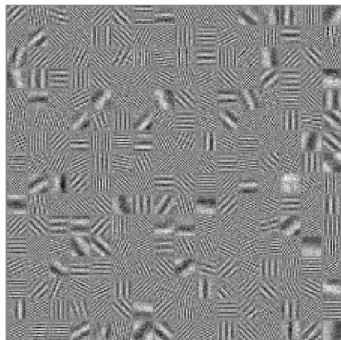
Extension to more than two frames



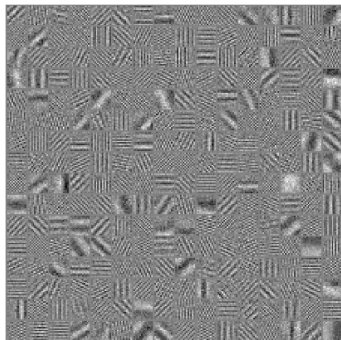
Extension to more than two frames



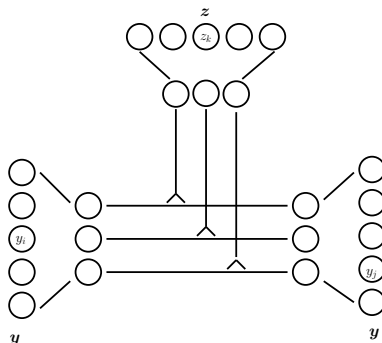
Extension to more than two frames



Extension to more than two frames

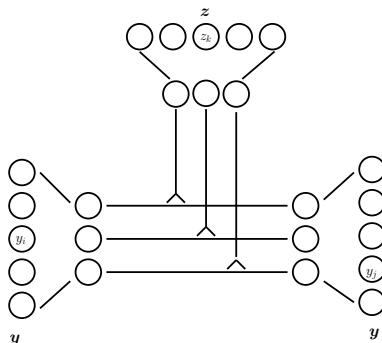


Extension to *less than two frames*



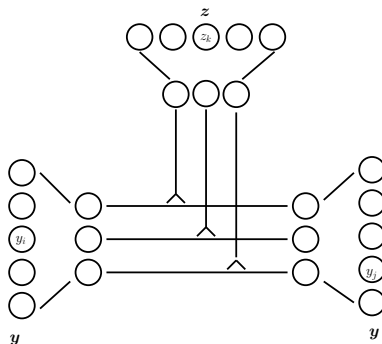
- To train energy models on *single images*:
- Plug in the same image left and right.
- Hiddens will model pixel covariance matrices.
- Eg., (Ranzato et al., 2010), (Karklin, Lewicki; 2008)
- Training can be finicky.

Extension to *less than two frames*



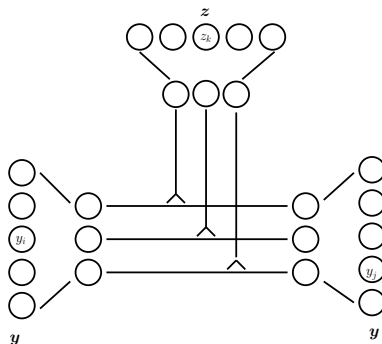
- To train energy models on *single images*:
- Plug in the same image left and right.
- Hiddens will model pixel covariance matrices.
- Eg., (Ranzato et al., 2010), (Karklin, Lewicki; 2008)
- Training can be finicky. Use a relational auto-encoder.

Extension to *less than two frames*



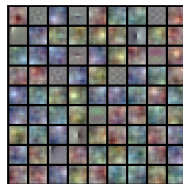
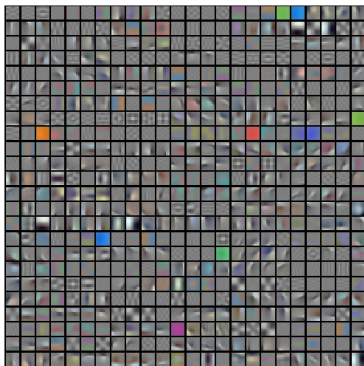
- Can also combine this with standard hidden units in one model.
- The combination tends to work better recognition (Ranzato et al., 2010).
- The vanilla hidden units then plays the role of “higher-order-biases” (Memisevic, 2007).

Extension to *less than two frames*



- Learning higher-order within-image structure has been suggested to address the fact that ICA does not really yield independent components...
- Add layers to model correlations of filter responses.
- Closely related to Deep Learning.

Some within image covariance and mean filters



Take-home message, factored model

To gather evidence for a transformation,
let each hidden unit **pool over products** of filter responses.

A bag of tricks

Tricks for learning:

- Normalize filters during learning, so they *grow slowly*, and they *grow together*. Normalize with a running average of the average filter norms.
- Connect top-level hidden *locally* to the factors.
- Probably even better: make them *locally overlapping* (“Topographic ICA”).
- *DC-centering and contrast-normalization* for each patch.
- Plus: *Whiten the data* before learning, using PCA or ZCA.
- Fast learning: large data-sets essential (use GPU’s...).

A bag of tricks

Tricks for learning:

- Normalize filters during learning, so they *grow slowly*, and they *grow together*: Normalize with a running average of the average filter norms.
- **Connect top-level hidden *locally* to the factors.**
- Probably even better: make them *locally overlapping* (“Topographic ICA”).
- *DC-centering and contrast-normalization* for each patch.
- Plus: *Whiten the data* before learning, using PCA or ZCA.
- Fast learning: large data-sets essential (use GPU’s...).

A bag of tricks

Tricks for learning:

- Normalize filters during learning, so they *grow slowly*, and they *grow together*. Normalize with a running average of the average filter norms.
- Connect top-level hidden *locally* to the factors.
- **Probably even better: make them *locally overlapping* (“Topographic ICA”).**
- *DC-centering and contrast-normalization* for each patch.
- Plus: *Whiten the data* before learning, using PCA or ZCA.
- Fast learning: large data-sets essential (use GPU’s...).

A bag of tricks

Tricks for learning:

- Normalize filters during learning, so they *grow slowly*, and they *grow together*: Normalize with a running average of the average filter norms.
- Connect top-level hidden *locally* to the factors.
- Probably even better: make them *locally overlapping* (“Topographic ICA”).
- *DC-centering and contrast-normalization for each patch.*
- Plus: *Whiten the data* before learning, using PCA or ZCA.
- Fast learning: large data-sets essential (use GPU’s...).

A bag of tricks

Tricks for learning:

- Normalize filters during learning, so they *grow slowly*, and they *grow together*: Normalize with a running average of the average filter norms.
- Connect top-level hidden *locally* to the factors.
- Probably even better: make them *locally overlapping* (“Topographic ICA”).
- *DC-centering and contrast-normalization* for each patch.
- **Plus: Whiten the data before learning, using PCA or ZCA.**
- Fast learning: large data-sets essential (use GPU’s...).

A bag of tricks

Tricks for learning:

- Normalize filters during learning, so they *grow slowly*, and they *grow together*: Normalize with a running average of the average filter norms.
- Connect top-level hidden *locally* to the factors.
- Probably even better: make them *locally overlapping* (“Topographic ICA”).
- *DC-centering and contrast-normalization* for each patch.
- Plus: *Whiten the data* before learning, using PCA or ZCA.
- **Fast learning: large data-sets essential (use GPU’s...).**

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

- Applications
- Conclusions

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

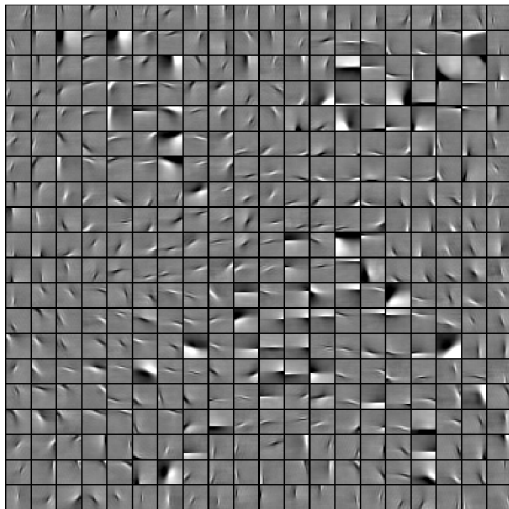
3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

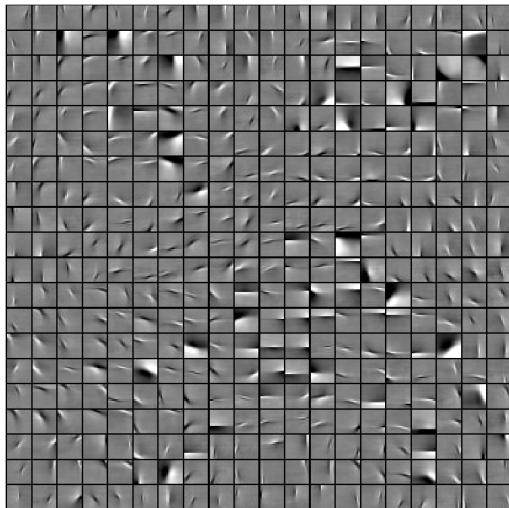
4 Applications

- Applications
- Conclusions

Bag-Of-Warps



Bag-Of-Warps



KTH Actions dataset



- Collapsing all hidden representations at monocular SIFT keypoints (across all keypoints and time frames) and performing logistic regression yields 80.56% correct.

Convolutional GBM

- Convolutional GBM:

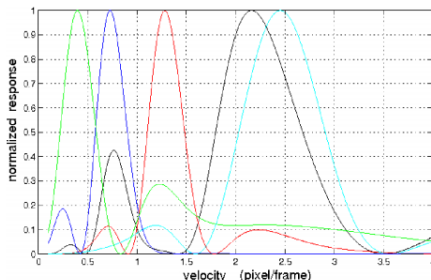
Prior Art	Accuracy	Convolutional architectures	Accuracy
HOG3D-KM-SVM	85.3	32convGRBM ^{16x16} -128F ^{9x9x9} -R/N/P ^{4x4x4} -log_reg	88.9
HOG/HOF-KM-SVM	86.1	32convGRBM ^{16x16} -128F ^{9x9x9} -R/N/P ^{4x4x4} -mlp	90.0
HOG-KM-SVM	79.0	32F ^{16x16x2} _{CSG} -R/N/P ^{4x4x4} _A -128F ^{9x9x9} _{CSG} -R/N/P ^{4x4x4} _A -log_reg	79.4
HOF-KM-SVM	88.0	32F ^{16x16x2} _{CSG} -R/N/P ^{4x4x4} _A -128F ^{9x9x9} _{CSG} -R/N/P ^{4x4x4} _A -mlp	79.5

- Convolutional GBM on Hollywood2:

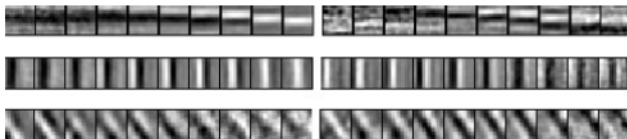
Method	AP
Prior Art [27]:	
HOG3D+KM+SVM	45.3
HOG/HOF+KM+SVM	47.4
HOG+KM+SVM	39.4
HOF+KM+SVM	45.5
convGRBM+SC+SVM	46.6

Stacked convolutional ISA

- (Le, et al., 2011)
- Velocity tuning of the higher-order features:



ISA applied to action recognition



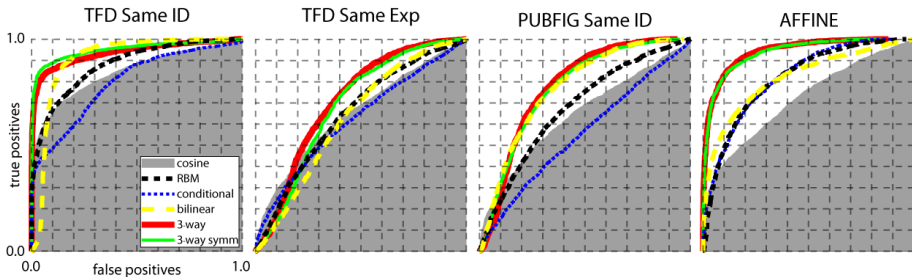
- (Le, et al., 2011)

	KTH	Hollywood2	UCF	YouTube
until 2011	92.1	50.9	85.6	71.2
hierarchical ISA	93.9	53.3	86.5	75.8

Metric learning

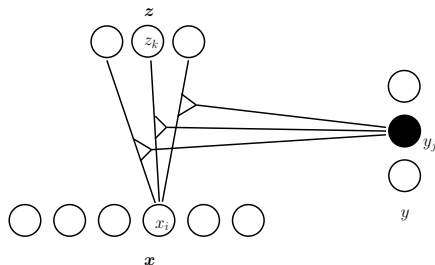


- Learning a gated Boltzmann machine on changing facial expressions.
- (Susskind, et al., 2011)
- Joint training allows for comparing compatibilities of pairs.



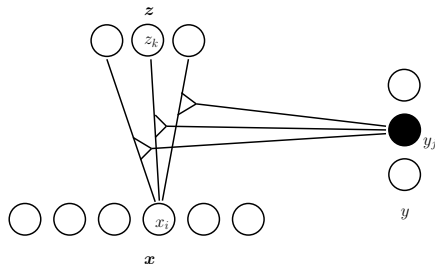
Model/Task	TFD ID	TFD Exp	PUBFIG ID	AFFINE
cosine	0.848	0.663	0.649	0.721
RBM	0.869	0.656	0.647	0.799
conditional	0.805	0.634	0.557	0.825
bilinear	0.905	0.637	0.774	0.812
3-way	0.932	0.705	0.771	0.930
3-way symm	0.951	0.695	0.762	0.931

Bi-linear classification



- Special case of a gated Boltzmann machine:
- Let output-“image” be one-hot-encoded **class-label**.
- This is a classifier, where each *label can blend in it's own model!*
- It is also equivalent to a mixture of 2^K logistic regressors (Memisevic, et al.; 2010)
- Boosting variant: (Warrell et al.; 2010)

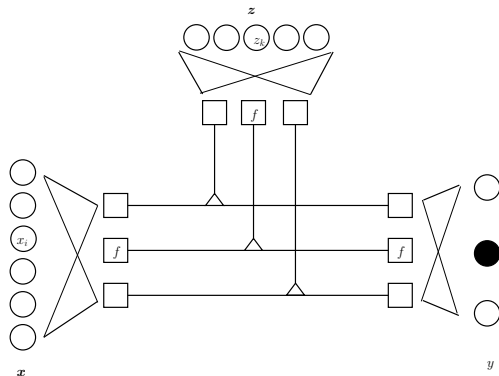
Bi-linear classification



- Marginalization is tractable in closed form

$$\begin{aligned} p(y|\mathbf{x}) &= \sum_{\mathbf{z}} p(y, \mathbf{z}|\mathbf{x}) \propto \sum_{\mathbf{z}} \exp(\mathbf{x}^t W_y \mathbf{z}) \\ &= \sum_{\mathbf{z}} \exp\left(\sum_{ik} W_{yik} x_i h_k\right) \\ &= \prod_k \left(1 + \exp\left(\sum_i W_{yik} x_i\right)\right) \end{aligned}$$

Bi-linear classification



- Factorization possible like before.
- Allows classifiers to share features.
- “2011 version of Hinton’s 1981 invariant recognition.”

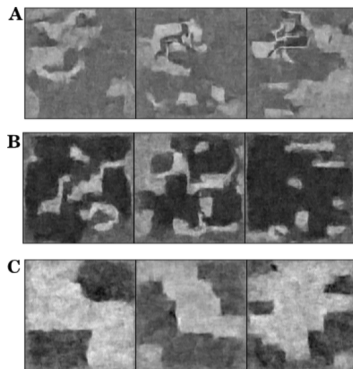
Bi-linear classification

- Letting labels act like gates.
- Deep Learning challenge (Larochelle et al., 2008).

	SVMs		NNet	RBM	DEEP		GSM	
dataset/model:	SVMRBF	SVMPOL	NNet	DBN1	DBN3	SAA3	GSM	(unfact)
rectangles	2.15	2.15	7.16	4.71	2.60	2.41	0.83	(0.56)
rect.-images	24.04	24.05	33.20	23.69	22.50	24.05	22.51	(23.17)
mnistplain	3.03	3.69	4.69	3.94	3.11	3.46	3.70	(3.98)
convexshapes	19.13	19.82	32.25	19.92	18.63	18.41	17.08	(21.03)
mnistbackrand	14.58	16.62	20.04	9.80	6.73	11.28	10.48	(11.89)
mnistbackimg	22.61	24.01	27.41	16.15	16.31	23.00	23.65	(22.07)
mnistrotbackimg	55.18	56.41	62.16	52.21	47.39	51.93	55.82	(55.16)
mnistrot	11.11	15.42	18.11	14.69	10.30	10.30	11.75	(16.15)

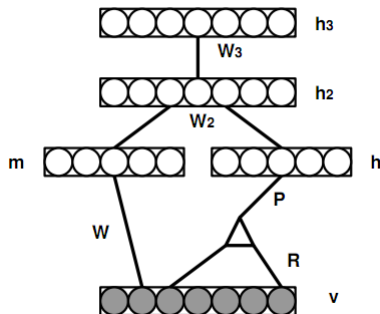
Within-image correlations

- (Karklin, Lewicki; 2008), (Osindero et al., 2006), ...
- ISA itself used mainly for modeling within-image structure.
- (Ranzato et al., 2010):



mcRBMs on TIMIT

- mcRBM applied to speech recognition (phones, speaker independent, TIMIT)
- (Dahl, et al.; 2010)



Method	PER
Stochastic Segmental Models [17]	36%
Conditional Random Field [18]	34.8%
Large-Margin GMM [19]	33%
CD-HMM [20]	27.3%
Augmented conditional Random Fields [20]	26.6%
Recurrent Neural Nets [21]	26.1%
Bayesian Triphone HMM [22]	25.6%
Monophone HTMs [23]	24.8%
Heterogeneous Classifiers [24]	24.4%
Deep Belief Networks(DBNs) [5]	23.0*%
Triphone HMMs discriminatively trained w/ BMML [7]	22.7%
Deep Belief Networks with mcRBM feature extraction (this work)	21.7*%
Deep Belief Networks with mcRBM feature extraction (this work)	20.5%

- (Taylor, et al.; 2010)



Training	Test	Baseline	MoCorr [28]	GPLVM [13]	CMFA-VB [13]	CRBM	imCRBM-10
S1+S2+S3	S1	129.18±19.47	140.35	-	-	55.43±0.79	54.27±0.49
S1	S1		-	-	-	48.75±3.72	58.62±3.87
S1+S2+S3	S2	162.75±15.36	149.37	-	-	99.13±22.98	69.28±3.30
S2	S2		-	88.35±25.66	68.67±24.66	47.43±2.86	67.02±0.70
S1+S2+S3	S3	180.11±24.02	156.30	-	-	70.89±2.10	43.40±4.12
S3	S3		-	87.39±21.69	69.59±22.22	49.81±2.19	51.43±0.92

1 Introduction

- Feature Learning
- Correspondence in Computer Vision
- Relational feature learning

2 Learning relational features

- Sparse Coding Review
- Encoding relations
- Inference
- Learning

3 Factorization, eigen-spaces and complex cells

- Factorization
- Eigen-spaces, energy models, complex cells

4 Applications

- Applications
- Conclusions

- Learning features that encode relations.
- Can address *correspondences* in vision (and beyond) through learning.
- *This seems like a very good reason to have complex cells.*
- Future work: More applications, more domains; Use as a building block in deeper models; Faster Learning; Deeper understanding of theoretical underpinnings; etc.

The end

- More info, code, links, etc. at

`http://www.cs.toronto.edu/~rfm/relational/index.html`